

교과목 3 : 초거대언어모델(LLM)

단원 4 : 프롬프트 엔지니어링

DAY4. Vector DB와 LLM을 결합한 RAG 아키텍처

목 차

1. RAG 개념		3
2. embedding		27
3. RAG qa		50

1. RAG 개념

검색으로 근거를 찾아 답하기

승승장구몰의 환불 정책, 제품 매뉴얼 같은 사내 문서(PDF)의 내용은 LLM이 모릅니다. 그래서 답이 있는 문서를 먼저 검색해 그 내용을 근거로 답하게 합니다. 이게 **RAG(Retrieval-Augmented Generation, 검색 증강 생성)**입니다.

1. 도구로도 못 푸는 질문

고객이 이렇게 묻습니다.

"단순 변심으로 반품하면 환불이 며칠 걸려요? 그리고 왕복 배송비는 누가 내요?"

이건 실시간 DB 조회(주문·재고)로 풀 문제가 아닙니다. 답은 사내 정책 문서([환불교환정책.pdf](#))에 있습니다.

```
ask("단순 변심 환불 시 배송비는 누가 내나요?")
# → LLM이 그럴듯하게 지어냄 (환각!)
# 회사마다 정책이 다른데, LLM이 우리 회사 정책을 알 리 없다.
```

2. "모르면, 답이 있는 곳을 찾아라"

해결책은 단순합니다. 답이 있는 문서를 먼저 찾아서, 그 내용을 근거로 답하게 하는 것입니다.

질문: "환불 시 배송비는?"

1. 환불 정책 문서에서 관련 부분을 검색 → "단순 변심 시 왕복 배송비는 고객 부담"
2. 그 내용을 LLM에게 주며 "이걸 근거로 답하라"
3. 답변: "단순 변심 반품 시 왕복 배송비는 고객님의 부담입니다."

DB 조회 결과를 응답에 끼워 넣듯이, 문서 검색 결과를 프롬프트에 끼워 넣는 것 — 그게 RAG입니다.

비유: 시험을 볼 때, 외운 지식으로만 답하는 것(LLM 단독)과 오픈북으로 교재를 찾아 답하는 것(RAG)의 차이입니다. 오픈북이면 정확한 근거를 보고 답하니 틀릴 일이 줄죠. RAG는 LLM에게 "오픈북"을 쥐여 주는 겁니다.

3. RAG라는 이름 풀이

RAG = Retrieval-Augmented Generation

- **Retrieval(검색)**: 답이 있을 법한 문서 조각을 찾는다
- **Augmented(증강)**: 찾은 내용을 프롬프트에 더한다(보강한다)
- **Generation(생성)**: 그 근거로 답을 생성한다

한 줄로: "검색해서(R) 찾은 근거로 보강해서(A) 답을 생성한다(G)."

구분	웹 검색	RAG
검색 대상	인터넷 전체	우리 사내 문서
신뢰성	낮음(광고·낙시 섞임)	높음(우리가 관리하는 문서)
검색 방식	키워드 검색	의미 기반 검색(임베딩)

RAG는 우리가 통제하는 신뢰할 수 있는 문서를 대상으로 하고, 의미 기반으로 검색합니다. 그래서 더 정확하고 믿을 만합니다.

핵심 정리

- 사내 문서(PDF) 내용은 LLM이 모른다 → 그냥 물으면 환각.
- **RAG** = 답이 있는 문서를 검색해 근거로 삼아 답을 생성.
- 웹 검색과 달리 신뢰할 수 있는 사내 문서를 의미 기반으로 검색.

LLM의 세 가지 한계

LLM에는 세 가지 한계가 있습니다 — 지식 컷오프, 환각, 사내 문서 모름.

RAG는 이 세 가지를 모두 외부 문서 검색으로 보완합니다. 핵심은 "가중치에 든 지식 대신, 외부에서 찾아온 근거로 답한다"입니다.

1. 세 가지 한계와 RAG의 해결

한계	설명	RAG의 해결
지식 컷오프	학습 시점 이후 정보를 모름	최신 문서를 검색해 프롬프트에 주입
환각 (Hallucination)	모르는 걸 그럴듯하게 지어냄	"주어진 문서 근거로만 답하라"로 통제
사내 문서 모름	우리 회사 정책·매뉴얼은 학습된 적 없음	사내 PDF/DB를 검색 대상으로 등록

하나씩 봅시다.

2. 한계 ① — 지식 컷오프

LLM은 학습 시점 이후를 모릅니다.

```
ask("2026년에 새로 나온 우리 신제품 사양은?")  
# → 학습 이후라 모름
```

RAG의 해결: 최신 제품 매뉴얼 PDF를 검색 대상으로 등록하면, 그 문서를 찾아 근거로 답합니다.

3. 한계 ② — 환각

LLM은 모르는 것도 그럴듯하게 지어냅니다.

```
ask("환불 시 배송비 정책 알려줘")
# → "보통 무료입니다" 같은 그럴듯한 거짓 (우리 회사 정책과 다를 수 있음)
```

RAG의 해결: "주어진 문서 내용만 근거로 답하고, 문서에 없으면 모른다고 하라"고 지시합니다 (16강에서 구현). 그러면 LLM이 함부로 지어내지 못하고, **문서에 적힌 사실만** 말합니다. 환각의 가장 강력한 방어책이죠.

4. 한계 ③ — 사내 문서 모름

가장 중요한 한계입니다. LLM은 우리 회사의 정책·매뉴얼을 학습한 적이 없습니다. 당연하죠 — 우리 사내 문서는 인터넷에 공개되지 않았으니까요.

```
ask("승승장구몰 VIP 멤버십 혜택은?")
# → 우리 회사 멤버십 정책을 알 리 없음 → 일반론이나 환각
```

RAG의 해결: `멤버십정책.pdf`를 검색 대상으로 등록하면, LLM이 그 문서를 찾아 우리 회사의 진짜 정책으로 답합니다. 이게 RAG가 기업에서 가장 많이 쓰이는 이유입니다 — **사내 지식을 LLM에게 연결하는 거죠.**

5. 핵심 아이디어 — 지식의 출처를 바꾼다

세 한계의 공통 해결책은 하나입니다.

"파라미터(가중치)에 든 지식 대신, 외부 저장소에서 찾아온 근거로 답한다."

LLM의 지식은 학습 때 **가중치(파라미터)**에 고정됩니다 — 바꿀 수 없고, 오래되고, 우리 문서는 없습니다. RAG는 답의 출처를 **외부 문서**로 옮깁니다.

```
LLM 단독: 질문 → [가중치 속 지식] → 답 (오래됨 · 환각 · 사내문서 없음)
RAG:      질문 → [외부 문서 검색] → 근거 → 답 (최신 · 정확 · 사내문서 포함)
```

외부 문서는 언제든지 업데이트할 수 있고, 우리가 관리하고, 출처를 확인할 수 있습니다. 그래서 더 정확하고 믿을 만하죠.

6. RAG가 바꾸는 것

RAG를 쓰면 LLM이 "박학다식하지만 우리 회사는 모르는 외부인"에서 "우리 회사 문서를 다 읽은 직원"으로 바뀝니다.

- 문서를 추가하면 → 새 지식을 얻음 (재학습 불필요!)
- 문서를 수정하면 → 즉시 반영됨
- 답의 출처를 → 문서로 추적 가능 (16강)

이게 기업이 RAG에 열광하는 이유입니다. 거대한 모델을 재학습하지 않고도, 문서만 관리하면 LLM이 최신 사내 지식으로 답하니까요.

핵심 정리

- LLM의 세 단계: 지식 컷오프 · 환각 · 사내 문서 모름.
- RAG는 셋 다 외부 문서 검색으로 보완한다.
- 핵심: 답의 출처를 가중치 → 외부 문서로 옮긴다.
- 외부 문서는 업데이트·관리·출처 추적이 가능해 더 믿을 만하다.

RAG 파이프라인 5단계

RAG는 다섯 단계로 돕니다: ① 로드 → ② 청킹 → ③ 임베딩 → ④ 저장/검색 → ⑤ 생성. 앞 4 단계는 사전 준비(인덱싱), 검색·생성은 매 질문마다(질의) 일어납니다.

1. 전체 그림

RAG는 두 시점으로 나뉩니다.

- ■ 사전 준비 (인덱싱 — 서비스 시작 전 1회/배치)
 - ① 로드(PDF/CSV → Document) → ② 청킹(긴 문서를 작은 조각으로) → ③ 임베딩(각 조각을 벡터로) → ④ 저장(벡터DB에 보관)
- ■ 질의 (매 질문마다)
 - 사용자 질문을 임베딩 → ④' 검색(벡터DB에서 유사 조각 k개) → ⑤ 생성(LLM이 그 조각을 근거로 답변)

2. 다섯 단계 해설

단계	이름	하는 일
①	로드(Load)	PDF 등을 Document로 읽기
②	청킹(Chunk)	긴 문서를 조각으로 나누기
③	임베딩(Embedding)	조각을 벡터로 변환
④	저장(Store)	벡터DB에 보관
⑤	생성(Generate)	검색한 근거로 답변

3. 두 단계로 나뉜 흐름 — 인덱싱 vs 질의

RAG의 5단계는 두 시점으로 나뉩니다. 이걸 이해하는 게 핵심입니다.

인덱싱 (①~④) — 사전 준비

서비스를 시작하기 전에 한 번 합니다. 문서를 로드·청킹·임베딩해서 벡터DB에 저장해 둡니다. 마치 도서관에서 책을 미리 분류해 서가에 꽂아 두는 것처럼요.

질의 (④'~⑤) — 매 요청

사용자가 질문할 때마다 합니다. 질문을 임베딩해서 유사한 조각을 검색하고, 그 조각을 근거로 답을 생성합니다. 도서관에서 "이 주제 책 어디 있어요?"라고 물으면 서가에서 찾아 주는 것처럼요.

백엔드 개발자 비유: 인덱싱은 "마이그레이션/시드"(서비스 전 1회), 질의는 "API 핸들러"(요청마다)라고 생각하면 깔끔합니다.

4. 왜 인덱싱을 미리 하나

"질문할 때마다 로드·청킹·임베딩을 다 하면 안 되나?"라고 생각할 수 있습니다. 안 됩니다.

- **느림**: 100페이지 PDF를 매 질문마다 로드·임베딩하면 답이 몇 초씩 걸립니다.
- **비용**: 임베딩은 API 호출(비용)입니다. 매번 전체를 임베딩하면 돈이 낭비됩니다.

그래서 무거운 작업(①~④)은 **미리 한 번 해 두고**, 가벼운 작업(검색·생성)만 **매번** 합니다. 검색은 미리 만든 벡터DB를 조회하는 거라 빠르죠. 이 분리가 RAG를 실용적으로 만듭니다.

핵심 정리

- RAG 5단계: **로드** → **청킹** → **임베딩** → **저장/검색** → **생성**.
- 두 시점: **인덱싱**(①~④, 사전 1회) + **질의**(④'~⑤, 매 요청).
- 무거운 작업은 **미리**, 가벼운 검색·생성만 **매번** → 빠르고 싸다.

왜 청킹하는가

긴 PDF를 통째로 쓰면 안 됩니다. **토큰·비용이 폭발**하고, 질문과 무관한 99%가 섞여 **검색 정확도가 떨어집니다**. 그래서 문서를 의미가 또렷한 **작은 조각(청크)**으로 나눕니다. 조각이 곧 검색 단위입니다.

1. 통째로 넣으면 안 되는 세 가지 이유

100페이지 매뉴얼을 통째로 프롬프트에 넣으면 어떻게 될까요?

① 토큰 한계·비용

2강에서 배웠듯 토큰이 곧 비용입니다. 100페이지를 **매 질문마다 통째로 넣으면** 토큰이 폭발하고, 모델의 입력 한계를 넘을 수도 있습니다.

② 검색 정확도 (Signal vs Noise)

"환불 며칠 걸려요?"라는 질문에 100페이지 전체를 주면, 질문과 **무관한 99%**가 같이 들어갑니다. LLM이 그 속에서 답을 찾느라 헛갈리고, 엉뚱한 부분을 참고할 수 있습니다.

③ 검색 단위

임베딩·검색은 "조각 단위"로 합니다. 조각이 적당히 작아야 의미가 뚜렷해 검색이 정확합니다.

비유: 도서관에서 "환불 정책"을 찾는데, 사서가 책 한 권 전체를 던져 주면(통째) 그 안에서 또 찾아야 합니다. 하지만 해당 페이지만 펴서 주면(청킹) 바로 답을 봅니다. 청킹은 "필요한 페이지만 뽑는" 작업입니다.

2. 청킹이란?

청킹(chunking) 은 긴 문서를 작은 조각(청크, chunk)으로 나누는 것입니다.

예를 들어 `환불교환정책.pdf` (3페이지)를 청킹하면, 작은 조각 여러 개로 나눕니다.

청크1: 환불 규정... · 청크2: 교환 규정... · 청크3: 배송비 부담... · ... (총 9개 조각)

각 청크는 하나의 검색 단위가 됩니다. 질문이 오면 9개 조각 중 가장 관련 있는 것만 골라 LLM에게 줍니다. 100페이지가 아니라 관련된 1~2개 조각만요.

3. 청킹의 핵심 파라미터

청킹에는 두 가지 핵심 설정이 있습니다.

파라미터	의미	권장값(한국어 정책문서)
<code>chunk_size</code>	한 조각의 최대 글자 수	500
<code>chunk_overlap</code>	인접 조각이 겹치는 글자 수	50

`chunk_size` — 조각 크기

한 조각을 몇 글자로 자를지입니다. 너무 크면 무관한 내용이 섞이고, 너무 작으면 맥락이 끊깁니다. 500자가 한국어 정책 문서에 무난한 출발점입니다.

chunk_overlap — 겹침

인접한 조각을 조금 겹치게 자르는 양입니다. 왜 겹칠까요?

4. chunk_overlap이 필요한 이유

문장이 조각 경계에서 뚝 잘리면 맥락이 끊깁니다.

겹침 없이 자르면:

청크1: "...환불은 상품 수령 후 7일 이내 신청 가능하며 왕복"

청크2: "배송비는 고객이 부담합니다..."

→ "왕복 배송비는 고객 부담"이 두 조각으로 쪼개져, 검색 시 의미가 분산!

겹쳐서 자르면 (overlap=50):

청크1: "...환불은 7일 이내 신청 가능하며 왕복 배송비는 고객이 부담합니다"

청크2: "왕복 배송비는 고객이 부담합니다. 교환의 경우..."

→ 경계에 걸친 정보가 양쪽에 다 들어가 보존됨!

`chunk_overlap` 은 조각 경계에 걸친 정보가 사라지지 않게 하는 안전장치입니다. 50자 정도 겹치면 문장 하나가 통째로 잘리는 일을 막을 수 있죠.

비유: 책을 여러 권으로 나눌 때, 각 권의 마지막 문단을 다음 권 첫머리에 조금 반복해 주면, 권이 바뀌어도 "이야기가 어디서 이어지는지" 알 수 있습니다. overlap이 그 역할입니다.

5. 청킹 도구 — RecursiveCharacterTextSplitter

LangChain은 `RecursiveCharacterTextSplitter`로 청킹합니다. 이름이 길지만 하는 일은 단순합니다.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,      # 한 조각 최대 500자
    chunk_overlap=50,    # 50자 겹침
)
chunks = splitter.split_documents(docs)
```

"Recursive(재귀적)"라는 이름은, **단락** → **문장** → **단어** 순으로 "자연스러운 경계"를 우선해 자른다는 뜻입니다. 그래서 문장 중간에서 툇 끊기는 일이 적습니다. 툇툇하게 자르는 거죠.

핵심 정리

- 긴 문서를 통째로 쓰면 **토큰 폭발·검색 부정확**.
- **청킹** = 문서를 작은 조각으로 나눠 검색 단위로 만들기.
- `chunk_size`(조각 크기)와 `chunk_overlap`(겹침)이 핵심 파라미터.
- `chunk_overlap`은 **경계에 걸친 정보**가 사라지지 않게 한다.
- `RecursiveCharacterTextSplitter`가 자연스러운 경계로 툇툇하게 자른다.

따라하기 — PDF 로드하기

목표

`PyPDFLoader`로 사내 PDF(환불교환정책.pdf)를 읽어, 페이지 단위 `Document` 객체로 만듭니다. 실제 파일은 `code/ch13_rag_concept.py`입니다.

0. 실습 준비 (설치)

```
conda activate agentic
```

```
# 13강에서 처음 쓰는 라이브러리
```

```
# - langchain-community : PyPDFLoader 등 문서 로더
```

```
# - langchain-text-splitters : RecursiveCharacterTextSplitter
```

```
# - pypdf : PDF 읽기 실제 엔진
```

```
pip install langchain langchain-community langchain-text-splitters pypdf
```

이번 강은 `data/docs/환불교환정책.pdf`를 씁니다. 임베딩 없이 진행하므로 API 키가 없어도 됩니다.

1. PyPDFLoader로 PDF 읽기

`PyPDFLoader`는 PDF를 페이지 단위로 읽어 `Document` 객체로 만듭니다.

```
# 실행: python code/ch13_rag_concept.py
```

```
import sys, pathlib
```

```
sys.path.append(str(pathlib.Path(__file__).resolve().parent))
```

```
from common import DOCS # DOCS = data/docs 폴더 경로
```

```
# PyPDFLoader: PDF를 페이지 단위 Document(본문+메타데이터)로 읽어 오는 로더
```

```
from langchain_community.document_loaders import PyPDFLoader
```

```
pdf_path = DOCS / "환불교환정책.pdf"
```

```
loader = PyPDFLoader(str(pdf_path))
```

```
docs = loader.load() # 페이지 단위 Document 리스트
```

`loader.load()`는 PDF의 각 페이지마다 `Document` 하나를 만듭니다. 3페이지 PDF면 `Document` 3개가 나오죠.

2. Document 구조 들여다보기

`Document` 는 두 부분으로 이뤄집니다.

```
print("로드한 페이지 수:", len(docs))
print("첫 페이지 메타데이터:", docs[0].metadata)
print("첫 페이지 내용 일부:")
print(docs[0].page_content[:200])
```

예상 출력

```
로드한 페이지 수: 3
첫 페이지 메타데이터: {'source': '.../환불교환정책.pdf', 'page': 0}
첫 페이지 내용 일부:
승승장구물 환불 · 교환 정책
1. 환불 규정
단순 변심에 의한 환불은 상품 수령 후 7일 이내 신청 가능합니다 ...
```

`Document` 의 두 부분:

- `page_content` : 페이지의 본문 텍스트 (실제 내용)
- `metadata` : 출처 정보 — `source` (파일 경로), `page` (페이지 번호)

3. metadata가 중요한 이유

`metadata` 의 `page` 정보는 단순히 보이지만 매우 중요합니다. 나중에 답변에 출처를 표시할 때 쓰이거든요(16강).

```
# 16강에서 이렇게 활용됨:
# "환불은 7일 이내 가능합니다. (출처: 환불교환정책.pdf 1페이지)"
# → 괄호 안 출처(파일명 · 페이지)는 Document.metadata 에서 가져온다.
```

RAG의 큰 장점 중 하나가 "답의 출처를 추적할 수 있다" 는 것인데(02번), 그게 바로 이 `metadata` 덕분입니다. 어느 문서 몇 페이지에서 나온 답인지 알 수 있죠. 그래서 로드 단계에서 `metadata`가 잘 보존되는 게 중요합니다.

4. Document — RAG의 공통 화폐

`Document` (`page_content` + `metadata`)는 RAG 파이프라인 전체에서 쓰이는 공통 데이터 형식입니다.

PDF 로드 → **Document** → 청킹 → **Document(조각)** → 임베딩 → 벡터

이처럼 `Document` 형식이 파이프라인 내내 흐릅니다(굵게 표시한 두 지점).

PDF든 CSV든 무엇을 로드하든, 결과는 항상 `Document` 형식입니다(08번에서 확인). 그래서 청킹·임베딩 같은 이후 단계는 소스 종류와 무관하게 똑같이 동작합니다. 일종의 "공통 화폐"인 셈이죠.

5. 파일이 없을 때 대비

실무 코드는 파일이 없을 때를 대비합니다.

```
def load_pdf(name: str):
    """data/docs 안의 PDF를 로드해 페이지 단위 Document 리스트를 반환."""
    pdf_path = DOCS / name
    if not pdf_path.exists():
        raise SystemExit(f"[파일 없음] {pdf_path}\n data/docs 폴더에 PDF가 있는지 확인하세요.")
    return PyPDFLoader(str(pdf_path)).load()
```

파일이 없으면 친절한 안내와 함께 종료합니다. "FileNotFoundError"라는 알 수 없는 에러보다, "이 경로에 PDF가 있는지 확인하세요"가 훨씬 도움이 되죠.

핵심 정리

- 설치: `pip install langchain langchain-community langchain-text-splitters pypdf`.
- `PyPDFLoader.load()`는 PDF를 **페이지 단위 Document**로 만든다.
- `Document` = **page_content**(본문) + **metadata**(출처·페이지).
- `metadata`의 `page` 정보가 나중에 **출처 표시**의 근거가 된다(16강).
- `Document`는 RAG 파이프라인의 **공통 형식**(소스 무관).

따라하기 — 청킹하기

목표

05번에서 로드한 Document를 `RecursiveCharacterTextSplitter`로 청킹합니다. 그리고 청크가 "색 단위"임을 눈으로 확인합니다.

1. 청킹 도구 준비

```
# RecursiveCharacterTextSplitter: 긴 문서를 적당한 크기 조각(청크)으로 나누는 도구
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,      # 한 조각 최대 500자
    chunk_overlap=50,    # 인접 조각 50자 겹침(경계에 걸친 맥락 보존)
)
```

파라미터(chunk_size, chunk_overlap)를 넣었습니다. 500자씩, 50자 겹치게 자르겠다는 설정이죠.

2. 청킹 실행

```
# split_documents: 원본 metadata(page 등)를 각 청크에 복사해 출처 추적이 가능하다
chunks = splitter.split_documents(docs)  # docs는 05번에서 로드한 Document 리스트

print("청크 개수:", len(chunks))
print("첫 청크 길이:", len(chunks[0].page_content))
print("첫 청크 내용:")
print(chunks[0].page_content)
print("첫 청크 메타데이터:", chunks[0].metadata)
```


예상 출력

```
청크 개수: 9
첫 청크 길이: 487
첫 청크 내용:
  승승장구물 환불 · 교환 정책
1. 환불 규정
단순 변심에 의한 환불은 상품 수령 후 7일 이내 ...
첫 청크 메타데이터: {'source': '.../환불교환정책.pdf', 'page': 0}
```

3페이지 PDF가 **9개** 청크로 나뉘었습니다. 각 청크는 약 500자이고요.

3. split_documents의 두 가지 똑똑함

`split_documents` 는 단순히 글자 수로 자르는 게 아닙니다. 두 가지를 똑똑하게 합니다.

① 자연스러운 경계로 자른다

`RecursiveCharacterTextSplitter` 는 단락 → 문장 → 단어 순으로 경계를 우선합니다. 그래서 "환불 규정은 다음과 같습니다"에서 똑 끊기지 않고, 가능한 한 문장·단락 끝에서 자릅니다.

② metadata를 복사한다

원본 Document의 `metadata` (page 등)를 각 청크에 복사합니다. 그래서 청크가 몇 페이지에서 왔는지 알 수 있죠.

```
print(chunks[0].metadata)  # {'source': '...', 'page': 0} ← 출처 보존!
```

이 덕분에 나중에 "이 답은 1페이지에서 나왔다"고 추적할 수 있습니다(16강).

4. 청크가 검색 단위임을 확인

청크가 정말 "검색 단위"인지 확인해 봅시다. "환불"이라는 단어가 들어간 청크를 찾아보겠습니다.

```
# [한계] 지금은 단어가 똑같이 들어간 청크만 찾는다. "돈 돌려받기"로는 못 찾는다.
keyword = "환불"
hits = [c for c in chunks if keyword in c.page_content]
print(f"'{keyword}'을 포함한 청크: {len(hits)}개")
for i, c in enumerate(hits[:2], 1):
    print(f"\n[{i}] (page {c.metadata.get('page')})")
    print(c.page_content[:120], "...")
```

예상 출력

'환불'을 포함한 청크: 4개

[1] (page 0)

승승장구물 환불 · 교환 정책 1. 환불 규정 단순 변심에 의한 환불은 상품 수령 후 7일 이내 ...

[2] (page 1)

환불 처리는 영업일 기준 3~5일 소요되며 ...

9개 청크 중 "환불"이 들어간 4개를 찾았습니다. 질문이 "환불 관련"이면 이 4개 중에서 답을 찾으면 되죠. 전체 9개가 아니라 관련된 것만 보는 겁니다.

5. 지금 방식의 한계 — 단어 매칭

위 코드는 단순 문자열 매칭입니다. "환불"이라는 단어가 똑같이 들어간 청크만 찾죠. 여기에 큰 한계가 있습니다.

```
# "환불"로 검색 → "환불" 들어간 청크 찾음 ✓
# "돈 돌려받기"로 검색 → "환불"과 같은 뜻인데 못 찾음! X
# "반품하면 결제 취소되나요?"로 검색 → 의미는 환불인데 못 찾음! X
```

사람은 "돈 돌려받기 = 환불"임을 알지만, 단어 매칭은 모릅니다. 글자가 다르면 못 찾죠.

이 한계를 푸는 게 **14강 임베딩(의미 기반 검색)**입니다. 임베딩을 쓰면 "돈 돌려받기"로도 "환불" 청크를 찾을 수 있습니다 — **단어가 아니라 의미로** 검색하니까요.

핵심 정리

- `split_documents(docs)`로 Document를 청크로 나눈다.
- `RecursiveCharacterTextSplitter`는 **자연스러운 경계**로 자르고 **metadata**를 복사한다.
- 각 청크는 하나의 **검색 단위** — 질문 관련 청크만 골라 쓴다.
- 지금은 **단어 매칭**이라 "돈 돌려받기"로 "환불"을 못 찾는 한계가 있다.
- 이 한계는 **임베딩(의미 검색)**으로 푼다.

청크 크기와 오버랩 튜닝

청크 설정(`chunk_size`, `chunk_overlap`)은 **검색 품질을 좌우하는 핵심 하이퍼파라미터**입니다. 여러 조합으로 청크 수·평균 길이·키워드 포함 청크 수를 비교해, 어떤 설정이 검색에 유리한지 감을 잡습니다.

1. "정답 설정"은 없다

06번에서 `(500, 50)`을 썼지만, 이건 출발점일 뿐 **정답이 아닙니다**. 문서 종류와 질문 패턴에 따라 최적 설정이 다릅니다.

- 조항이 짧은 정책 문서 → 작은 청크가 유리
- 맥락이 긴 설명서 → 큰 청크가 유리

그래서 여러 설정을 직접 **비교해 보며** 감을 잡아야 합니다. 이게 "튜닝(tuning)"입니다.

2. 여러 설정 비교하기

(실습 파일: [sections/13_rag_concept/07_청크-크기와-오버랩-튜닝.py](#)) — 임베딩 없이, 문자열 포함만으로 비교합니다.

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from common import DOCS

docs = PyPDFLoader(str(DOCS / "환불교환정책.pdf")).load()
keyword = "환불" # 같은 키워드가 포함된 청크 수로 '검색 명중률'을 가늠

# 비교할 (chunk_size, overlap) 조합들
configs = [(300, 0), (500, 50), (1000, 100)]

print(f"{'설정(size,overlap)':<22}{'청크수':>8}{'평균길이':>10}{'키워드포함':>12}")
for size, overlap in configs:
    splitter = RecursiveCharacterTextSplitter(chunk_size=size, chunk_overlap=overlap)
    chunks = splitter.split_documents(docs)
    n = len(chunks)
    avg_len = sum(len(c.page_content) for c in chunks) / n if n else 0
    hits = sum(1 for c in chunks if keyword in c.page_content)
    print(f"({size}, {overlap}){'':<10}{n:>8}{avg_len:>10.0f}{hits:>12}")
```

예상 출력

설정(size,overlap)	청크수	평균길이	키워드포함
(300, 0)	16	280	7
(500, 50)	9	450	4
(1000, 100)	5	720	3

3. 결과 읽기 — `chunk_size`의 영향

표를 보면 경향이 보입니다.

- `chunk_size`가 작을수록 (300):
 - 청크 수가 많아짐(16개), 평균 길이 짧아짐(280자)
 - → 조각이 잘게 쪼개져 "핀포인트" 검색에 유리
 - → 단, 맥락이 잘릴 위험 (한 조항이 여러 조각으로 분산)
- `chunk_size`가 클수록 (1000):
 - 청크 수가 적어짐(5개), 평균 길이 길어짐(720자)
 - → 한 청크에 맥락이 풍부
 - → 단, 무관한 내용도 함께 섞임 (signal vs noise)

비유: 작은 청크는 포스트잇 메모(짧고 핀포인트), 큰 청크는 한 페이지 요약(맥락 풍부하지만 군더더기 포함)입니다. 둘 다 장단이 있죠.

4. overlap의 역할

`chunk_overlap` 은 04번에서 배운 대로 **경계에 걸친 정보를 보존**합니다.

overlap=0 : 경계에서 문장이 잘리면 "왕복 배송비는 고객 부담"이 두 조각으로 분리
overlap=50 : 경계 부분이 양쪽 청크에 겹쳐 들어가 정보 보존

overlap이 클수록 정보 보존은 좋아지지만, 중복이 늘어 **저장 공간·임베딩 비용**이 약간 증가합니다. 보통 `chunk_size`의 10~20%(500이면 50~100)가 적당합니다.

5. "키워드 포함 수"의 함정

표에서 작은 청크일수록 "키워드 포함" 수가 큼니다($7 > 4 > 3$). 그렇다고 무조건 작은 게 좋은 건 아닙니다.

[경고] 한 답을 여러 청크로 쪼개면, 정작 검색 시 근거가 분산된다.

예를 들어 "환불 절차"가 원래 한 단락인데 4개 청크로 쪼개지면, 검색으로 1개만 찾았을 때 불완전한 답이 됩니다. 키워드가 많이 잡힌다고 좋은 게 아니라, **하나의 완결된 의미가 한 청크에 담기는 게 좋습니다.**

그래서 결론은: **(500, 50)**이 무난한 출발점이고, 문서를 보며 조정합니다. 정책 조항이 짧으면 좀 작게, 설명이 길면 좀 크게요.

6. 튜닝의 자세

청크 튜닝은 "한 번 정하고 끝"이 아닙니다.

1. 무난한 값(500, 50)으로 시작
2. 실제 질문으로 검색해 보고 답 품질 확인
3. 답이 잘리거나 부정확하면 → 청크 크기 조정
4. 반복

"오분류 분석 → 처방"과 같은 개선 순환입니다. RAG도 **측정하고 조정하며** 품질을 높입니다.

핵심 정리

- 청크 설정은 검색 품질을 좌우하는 하이퍼파라미터 — 정답은 없다.
- 작은 청크: 핀포인트 검색 유리, 맥락 잘림 위험.
- 큰 청크: 맥락 풍부, 무관 내용 섞임.
- overlap은 경계 정보 보존(보통 size의 10~20%).
- 키워드 많이 잡힌다고 좋은 게 아니다 — 완결된 의미가 한 청크에.
- (500, 50)에서 시작해 측정하며 조정한다.

다양한 문서 로더

RAG의 소스는 PDF만이 아닙니다. CSV(FAQ), 텍스트, 마크다운 등도 똑같이 Document로 바뀌어야 청킹·임베딩 파이프라인에 태울 수 있습니다. 핵심은 로더가 무엇이든 결과는 항상 같은 Document 구조라는 것입니다.

1. 문서는 PDF만 있는 게 아니다

승승장구물의 지식은 여러 형태로 흩어져 있습니다.

- PDF: 환불 정책, 제품 매뉴얼 (05번에서 다룸)
- CSV: FAQ 목록 (`faq.csv`)
- 텍스트: 사내 공지, 메모
- 마크다운: 위키, 문서

이 모든 걸 RAG에 쓰려면, 각각을 Document 형식으로 변환해야 합니다. LangChain은 형식마다 전용 로더를 제공합니다.

2. CSVLoader — 한 행 = 한 Document

`CSVLoader`는 CSV의 각 행을 **Document** 하나로 만듭니다. FAQ 같은 "한 문답 = 한 검색 단위" 데이터에 딱이죠. (실습 파일: `sections/13_rag_concept/08_다양한-문서-로더.py`)

```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader(file_path=str(DATA / "faq.csv"), encoding="utf-8-sig")
csv_docs = loader.load()
# faq.csv의 각 행(faq_id, question, answer)이 Document 하나가 된다
```

FAQ는 이미 "질문-답변"으로 잘게 나뉘어 있으니, 행 하나가 자연스러운 검색 단위입니다. 따로 청킹할 필요도 별로 없죠.

3. TextLoader — 일반 텍스트

`TextLoader`는 `.txt` 파일을 통째로 읽어 Document 하나로 만듭니다.

```
from langchain_community.document_loaders import TextLoader

text_docs = TextLoader("notice.txt", encoding="utf-8").load()
```

사내 공지, 메모 같은 단순 텍스트에 씁니다. 길면 청킹해서 나눠 쓰고요.

4. MarkdownHeaderTextSplitter — 마크다운

마크다운은 **헤더(#)** 단위로 나누면 의미 경계가 또렷합니다. `MarkdownHeaderTextSplitter`가 헤더 기준으로 나눠 줍니다.


```

from langchain_text_splitters import MarkdownHeaderTextSplitter

markdown = (
    "# 환불 정책\n구매 후 7일 이내 단순 변심 환불이 가능합니다.\n"
    "# 교환 정책\n상품 불량 시 무상 교환해 드립니다."
)

splitter = MarkdownHeaderTextSplitter(headers_to_split_on=["#", "섹션"]))
md_docs = splitter.split_text(markdown)
# "# 환불 정책" 섹션과 "# 교환 정책" 섹션이 각각 Document가 된다

```

마크다운은 이미 헤더로 구조가 나뉘어 있으니, 그 구조를 살려 나누면 의미 단위로 깔끔하게 청킹됩니다.

5. 핵심 — 모두 같은 Document 구조

가장 중요한 점입니다. 로더가 무엇이든, 결과는 항상 같은 **Document** 구조입니다.

```

def show_docs(title, docs):
    for d in docs[:2]:
        print("page_content:", d.page_content[:80])
        print("metadata      :", d.metadata)

# PDF, CSV, 텍스트, 마크다운 — 모두 page_content + metadata!
show_docs("CSV", csv_docs)
show_docs("텍스트", text_docs)
show_docs("마크다운", md_docs)

```

예상 출력

```

[CSV] page_content: faq_id: F01 / question: 배송 얼마나 걸려요? / answer: 평균 1~3일 ...
      metadata      : {'source': '.../faq.csv', 'row': 0}
[텍스트] page_content: 승승장구몰 사내 공지 여름 정기 휴무는 ...
      metadata      : {'source': '.../notice.txt'}
[마크다운] page_content: 구매 후 7일 이내 단순 변심 환불이 ...
      metadata      : {'섹션': '환불 정책'}

```

소스는 제각각이지만 모두 **page_content + metadata** 구조죠. 이게 "공통 화폐"입니다.

6. 왜 이게 강력한가

모든 소스가 같은 `Document` 형식으로 통일되니, 이후 파이프라인을 재사용할 수 있습니다.

PDF · CSV · 텍스트 · 마크다운 → (각 로더로) → **Document** → 청킹 → 임베딩 → 저장 → 검색
뒷부분(청킹~검색)은 소스 종류와 무관하게 동일하므로, 한 번만 만들면 모든 소스에 재사용됩니다.

로더만 소스에 맞게 바꾸면, 청킹·임베딩·검색 코드는 그대로 씁니다. 새 문서 형식이 추가돼도 로더만 추가하면 되죠. 이게 잘 설계된 파이프라인의 힘입니다.

비유: 다양한 나라 화폐(PDF·CSV·텍스트)를 공항 환전소(로더)에서 모두 같은 통화(Document)로 바꾸면, 그 다음부터는 화폐 종류를 신경 쓸 필요가 없습니다. 한 가지 통화로만 거래하면 되니까요.

7. 미설치 대비

여러 로더는 추가 패키지가 필요할 수 있습니다. 실무 코드는 `try/except`로 감쌉니다.

```
try:
    from langchain_community.document_loaders import CSVLoader
    csv_docs = CSVLoader(file_path="...", encoding="utf-8-sig").load()
except Exception as e:
    print("CSVLoader 사용 불가:", e) # 데모가 죽지 않게
```

로더 하나가 안 돼도 전체 데모가 멈추지 않게 하는 거죠

핵심 정리

- RAG 소스는 PDF뿐 아니라 **CSV·텍스트·마크다운** 등 다양하다.
- CSVLoader(행 단위), TextLoader(텍스트), MarkdownHeaderTextSplitter(헤더 단위).
- 로더가 무엇이든 결과는 **항상 같은 Document 구조**(page_content + metadata).
- 그 덕에 **청킹·임베딩·검색 파이프라인을 재사용**할 수 있다.

2. embedding

텍스트를 벡터로, 의미를 거리로

검색은 "환불" in text 같은 단어 매칭이라, "돈 돌려받기"로는 "환불"을 못 찾았습니다.

텍스트를 의미를 담은 숫자 벡터(임베딩)로 바꾸면, 단어가 달라도 의미가 비슷하면 찾을 수 있습니다. 이게 RAG의 심장인 "의미 기반 검색"입니다.

1. 단어 매칭의 한계

1장에서 청크를 만들었지만, 검색은 단순 문자열 매칭이었습니다.

```
hits = [c for c in chunks if "환불" in c.page_content]
```

이건 한계가 명확합니다. 고객이 이렇게 물으면?

"돈 언제 돌려받아요?"

→ "환불"이라는 단어가 없음 → 매칭 실패! X

사람은 "돈 돌려받기 = 환불"임을 압니다. 하지만 단어 매칭은 글자가 같아야만 찾습니다. 의미가 같아도 글자가 다르면 못 찾죠.

2. 기계에게 '의미'를 가르치려면

기계가 "돈 돌려받기"와 "환불"이 비슷한 의미임을 알게 하려면 어떻게 할까요?

핵심 아이디어: 텍스트를 "의미를 담은 숫자"로 바꾸고, 그 숫자들이 비슷하면 의미도 비슷하다고 본다.

- "환불" → [0.021, -0.118, 0.073, ...]
- "돈 돌려받기" → [0.019, -0.121, 0.069, ...] — 숫자가 비슷함 → 의미도 비슷!
- "로봇청소기" → [-0.087, 0.203, -0.011, ...] — 숫자가 완전 다름 → 의미도 다름

이렇게 텍스트를 의미를 담은 숫자 벡터로 바꾸는 게 임베딩(embedding)입니다.

백엔드 비유: 문자열 비교가 `WHERE text LIKE '%환불%'` 라면, 임베딩 검색은 "의미 좌표가 가까운 행을 거리순으로 정렬"하는 것에 가깝습니다. 글자가 아니라 **의미의 위치**로 찾는 거죠.

3. "의미를 거리로" 바꾼다

임베딩의 마법은 의미를 거리(distance)로 바꾼다는 것입니다.

의미 공간(상상):

환불 •

돈돌려받기 • ← 환불과 가까이 위치

• 로봇청소기 ← 멀리 위치

• 배터리

비슷한 의미는 **가까이**, 다른 의미는 **멀리** 위치합니다. 그러면 "이 질문과 의미가 가까운 문서"를 거리 계산으로 찾을 수 있죠. "환불" 근처에 있는 문서를 찾으면, "돈 돌려받기"로 물어도 "환불 정책"을 찾아냅니다.

핵심 정리

- 단어 매칭은 글자가 같아야 찾아 "돈 돌려받기 ≠ 환불"의 한계.
- 임베딩 = 텍스트를 의미를 담은 숫자 벡터로 변환.
- 비슷한 의미는 벡터도 비슷 → 의미를 거리로 바꿔 검색.
- 벡터 DB 없이 numpy 로 원리를 먼저 체득

임베딩이란

임베딩(embedding)은 임의의 텍스트를 고정 길이의 숫자 벡터로 바꾸는 것입니다. 핵심 성질은 "의미가 비슷한 문장은 벡터도 비슷한 방향"을 가리킨다는 것입니다. 이 강에서는 768 차원 벡터를 씁니다.

1. 임베딩 = 텍스트 → 고정 길이 벡터

임베딩 모델은 어떤 텍스트를 넣든 항상 같은 길이의 숫자 배열을 돌려줍니다.

```
"환불 규정" → [0.021, -0.118, 0.073, ..., 0.004] (숫자 768개)
"단순 변심 환불 절차" → [0.018, -0.115, 0.071, ..., 0.006] (숫자 768개)
"로봇청소기 배터리" → [-0.087, 0.203, -0.011, ..., -0.140] (숫자 768개)
```

문장이 짧은 길든, 결과는 고정 길이(여기선 768개 숫자)입니다. 이 숫자 배열이 그 텍스트의 "의미 좌표"인 셈이죠.

2. 768차원이란?

벡터의 길이(숫자 개수)를 차원(dimension)이라고 합니다. 이 과정에서는 768차원을 씁니다.

```
v = emb.embed_query("환불은 며칠 걸려요?")
print(len(v)) # 768
```

`gemini-embedding-001` 모델은 기본 3072차원을 지원하지만(768·1536·3072 중 선택 가능), 이 과정에서는 저장 공간·속도를 고려해 768차원으로 설정해 씁니다(`common.py`의 `output_dimensionality=768`).

"차원이 뭐지?"라고 어렵게 생각할 필요 없습니다. 그냥 "숫자 768개로 의미를 표현한다"고 보면 됩니다. 숫자가 많을수록(차원이 높을수록) 의미를 더 세밀하게 표현하지만, 저장·계산 비용도 늘어납니다. 768은 그 균형점입니다.

3. 핵심 성질 — 의미가 비슷하면 벡터도 비슷

임베딩의 가장 중요한 성질입니다.

의미가 비슷한 문장은 벡터 공간에서 가까운 방향을 가리킨다.

- "환불" → $[0.021, -0.118, 0.073, \dots]$
- "돈 돌려받기" → $[0.019, -0.121, 0.069, \dots]$ — 숫자 패턴이 비슷 → 같은 방향(의미 유사)
- "로봇청소기" → $[-0.087, 0.203, -0.011, \dots]$ — 다른 방향(의미 무관)

"환불"과 "돈 돌려받기"는 글자가 하나도 안 겹치지만, 임베딩 벡터는 비슷합니다. 모델이 학습 과정에서 "이 둘은 비슷한 맥락에서 쓰인다"는 걸 익혔기 때문이죠. 반면 "로봇청소기"는 전혀 다른 방향을 가리킵니다.

4. 어떻게 의미를 숫자로?

"어떻게 텍스트의 의미를 숫자로 바꾸지?"가 궁금할 수 있습니다. 정확한 원리는 복잡하지만, 직관은 이렇습니다.

임베딩 모델은 방대한 텍스트로 학습하면서, "비슷한 맥락에서 쓰이는 말은 비슷한 숫자로" 표현하도록 훈련됐습니다. "환불"과 "돈 돌려받기"가 비슷한 문장들에서 자주 등장하니, 모델이 둘을 가까운 벡터로 배치하는 거죠.

비유: 도서관에서 책을 주제별로 서가에 배치하는 것과 같습니다. "요리책"들은 한 구역에, "여행책"들은 다른 구역에 모입니다. 임베딩은 문장을 의미별 좌표에 배치하는 자동 분류기인 셈입니다. 주제가 비슷하면 가까운 자리에 놓이죠.

5. 그래서 무엇을 할 수 있나

텍스트를 의미 좌표(벡터)로 바꾸면, 여러 가지가 가능해집니다.

- 의미 검색: 질문과 가까운 문서 찾기 (RAG의 핵심)
- 유사도 측정: 두 문장이 얼마나 비슷한가
- 분류·군집: 비슷한 것끼리 묶기

이 강에서는 의미 검색에 집중합니다. "질문과 의미가 가까운 FAQ 찾기"를 만들죠. 그러려면 "두 벡터가 얼마나 가까운가"를 재는 방법이 필요한데, 그게 다음 절의 코사인 유사도입니다.

핵심 정리

- 임베딩 = 텍스트를 고정 길이 숫자 벡터로 변환(이 강은 768 차원).
- 핵심 성질: 의미가 비슷하면 벡터도 비슷한 방향.
- 모델이 학습으로 "비슷한 맥락의 말 = 비슷한 숫자"를 익혔다.
- 의미를 좌표로 바꾸면 검색·유사도·분류가 가능.

코사인 유사도

코사인 유사도(cosine similarity) 는 두 벡터가 "얼마나 같은 방향을 가리키는가"를 재는 척도입니다. 크기가 아니라 방향(각도) 만 봅니다. 값은 -1~1 이고, 1 에 가까울수록 의미가 비슷합니다.

1. 두 벡터의 '닮음'을 재기

02번에서 "의미가 비슷하면 벡터도 비슷한 방향"이라고 했습니다. 그럼 "얼마나 비슷한 방향인가"를 숫자로 재야겠죠. 그게 코사인 유사도입니다.

$$\text{코사인 유사도}(A, B) = (A \cdot B) / (|A| \cdot |B|)$$

- 분자 $A \cdot B$ = 두 벡터의 내적
- 분모 $|A| \cdot |B|$ = 각 벡터의 길이(크기)의 곱

공식이 복잡해 보이지만, 의미는 단순합니다 — "두 벡터 사이의 각도가 얼마나 작은가" 입니다.

2. 값의 의미

코사인 유사도는 -1에서 1 사이의 값입니다.

값	각도	의미
1.0	0° (같은 방향)	의미 매우 유사
0.0	90° (직각)	무관
-1.0	180° (반대 방향)	정반대

- 벡터 A와 B가 같은 방향을 가리키면 → $\cos = 1.0$ (의미 같음)
- 벡터 A와 C가 직각을 이루면 → $\cos = 0.0$ (무관)

실무에서는 보통 0~1 범위가 나오고(텍스트 임베딩은 음수가 드물), **0.7 이상이면 꽤 비슷**, **0.5 이하면 별 관계 없음** 정도로 봅니다.

3. 왜 '방향'만 보나 (크기 무시)

코사인 유사도는 벡터의 크기는 무시하고 방향만 봅니다. 왜일까요?

문장의 길이나 강조 정도에 따라 벡터의 크기가 달라질 수 있는데, 우리가 알고 싶은 건 "의미가 같은 방향인가" 입니다. 크기가 아니라고요.

"환불" → 짧은 문장, 작은 크기 벡터
"환불 절차를 자세히 알려주세요" → 긴 문장, 큰 크기 벡터
→ 크기는 다르지만 '방향(의미)'은 비슷!

크기를 무시하고 방향만 보니, 길이가 달라도 의미가 같으면 높은 유사도가 나옵니다. 이게 텍스트 검색에 코사인 유사도를 쓰는 이유입니다.

비유: 두 사람이 같은 방향을 가리키는지 볼 때, 팔 길이(크기)는 상관없고 가리키는 방향만 중요하죠. 키 큰 사람과 작은 사람이 같은 곳을 가리키면, 팔 길이가 달라도 "같은 방향"입니다.

4. 코드로 구현

코사인 유사도를 numpy로 직접 구현하면 공식이 그대로 보입니다.

```
import numpy as np

def cosine(a, b) -> float:
    """두 벡터의 코사인 유사도(-1 ~ 1). 1에 가까울수록 의미가 비슷하다."""
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    denom = np.linalg.norm(a) * np.linalg.norm(b) # 분모 = 각 벡터 길이의 곱
    return float(a @ b / denom) if denom else 0.0 # a @ b = 내적
```

- `a @ b`: 내적(dot product) — 분자
- `np.linalg.norm(a)`: 벡터 a의 길이(크기) — 분모용
- `if denom else 0.0`: 분모가 0(영벡터)이면 0으로 나누기를 방지

공식 $(A \cdot B) / (|A| |B|)$ 가 코드에 그대로 들어 있죠.

5. 의미 검색의 알고리즘

코사인 유사도가 있으면, 의미 검색은 4단계로 됩니다.

1. 모든 후보 문서를 미리 임베딩해 둔다 (사전 준비)
2. 사용자 질문을 임베딩한다 (질문이 올 때)
3. 질문 벡터 vs 각 문서 벡터의 코사인 유사도 계산
4. 유사도 높은 순으로 정렬해 상위 k개 반환

이게 RAG 검색의 핵심 알고리즘입니다. 13 강에서 본 "인덱싱(1) vs 질의(2~4)" 구분도 여기 있죠. 벡터 DB(FAISS)는 이 4 단계를 대량·고속으로 해 주는 도구일 뿐, 원리는 똑같습니다.

핵심 정리

- 코사인 유사도 = 두 벡터의 **방향(각도)** 닮음, 값 $-1 \sim 1$.
- 1 에 가까울수록 의미 유사, 0 이면 무관.
- 크기는 무시하고 **방향만** 보므로 문장 길이에 안 흔들림.
- 공식: $(A \cdot B) / (|A||B|)$ — numpy 로 직접 구현 가능.
- 의미 검색 = 후보 임베딩 → 질문 임베딩 → 유사도 정렬 → 상위 k.

임베딩 모델 vs 채팅 모델

지금까지 사용한 **채팅 모델**(gemini-2.5-flash)은 텍스트를 받아 텍스트(답변)를 냅니다.

임베딩 모델(gemini-embedding-001)은 텍스트를 받아 숫자 벡터를 냅니다. 둘은 용도·비용이 다른 별개의 모델입니다.

1. 두 모델 비교

	채팅 모델 (gemini-2.5-flash)	임베딩 모델 (gemini-embedding-001)
입력	텍스트	텍스트
출력	텍스트(답변)	숫자 벡터(768차원)
용도	생성·추론	검색·유사도·분류
비용	상대적 높음	매우 저렴

같은 텍스트를 입력받지만, **출력이 완전히 다릅니다**. 채팅 모델은 "답"을 만들고, 임베딩 모델은 "의미 좌표"를 만들죠.

2. 출력이 다르다

가장 큰 차이는 출력입니다.

```
# 채팅 모델 — 텍스트 답변
chat_llm.invoke("환불은 며칠 걸려요?")
# → "환불은 보통 영업일 기준 3~5일 정도 소요됩니다." (사람이 읽는 답)

# 임베딩 모델 — 숫자 벡터
emb.embed_query("환불은 며칠 걸려요?")
# → [0.0123, -0.0481, 0.0207, ...] (의미를 담은 768개 숫자)
```

채팅 모델의 출력은 사람에게 보여 줄 답이고, 임베딩 모델의 출력은 기계가 검색에 쓸 좌표입니다. 용도가 다르죠.

3. 둘은 협력한다 (RAG에서)

RAG에서는 두 모델이 함께 일합니다.

질문: "환불 정책 알려줘"

1. 임베딩 모델(검색·유사도 담당): 질문을 벡터로 → 비슷한 문서 검색
2. 채팅 모델(생성 담당): 찾은 문서를 근거로 답변 생성
3. 답변: "단순 번심 환불은 7일 이내 가능합니다."

- 임베딩 모델: "어느 문서가 관련 있나"를 찾음 (검색)
- 채팅 모델: 찾은 문서로 "답"을 만들 (생성)

각자 잘하는 일을 합니다. 임베딩은 검색에 특화됐고(빠르고 썸), 채팅은 생성에 특화됐죠(똑똑하지만 비쌌). 이 분업이 RAG의 효율을 만듭니다.

4. 임베딩이 싼 이유

임베딩 모델이 채팅 모델보다 훨씬 저렴한 이유가 있습니다.

- 채팅 모델: 단어를 하나씩 생성해야 함 (무거운 작업)
- 임베딩 모델: 텍스트를 한 번 읽고 벡터 하나 출력 (가벼운 작업)

생성은 단어를 차례로 만드느라 계산이 많지만, 임베딩은 "의미 좌표 한 번 계산"이라 가볍습니다. 그래서 수천 개 문서를 임베딩해도 비용이 적게 듭니다. RAG에서 많은 문서를 인덱싱할 수 있는 이유죠.

5. common.py의 get_embeddings

이 과정에서는 `common.py`의 `get_embeddings()`로 임베딩 모델을 가져옵니다.

```
from common import get_embeddings

emb = get_embeddings() # GoogleGenerativeAIEmbeddings (gemini-embedding-001, 768차원)
```

`get_chat`(채팅 모델)과 짝을 이루는 함수죠.

내부적으로 `GoogleGenerativeAIEmbeddings(model="models/gemini-embedding-001", output_dimensionality=768)`을 만들어 줍니다.

`get_chat` 처럼, 우리는 세부 설정을 신경 쓸 필요 없이 `get_embeddings()`만 부르면 됩니다.

핵심 정리

- 채팅 모델: 텍스트 → 텍스트(답변), 생성·추론용, 비쌈.
- 임베딩 모델: 텍스트 → 숫자 벡터, 검색·유사도용, 싼.
- RAG 에서 둘이 협력: 임베딩이 검색, 채팅이 생성.
- 임베딩이 싸서 많은 문서를 인덱싱할 수 있다.
- `get_embeddings()`로 임베딩 모델을 가져온다(`get_chat`의 짝).

따라하기 — 문장을 벡터로

목표

`get_embeddings()`로 문장을 벡터로 바꿔 봅니다. 단일 문장은 `embed_query`, 여러 문장은 `embed_documents` (배치)를 씁니다. 실제 파일은 `code/ch14_embedding.py` 입니다.

0. 실습 준비 (설치)

```
conda activate agentic

# 14강에서 처음 쓰는 라이브러리
# - langchain-google-genai : GoogleGenerativeAIEmbeddings (Gemini 임베딩)
# - numpy                  : 벡터 계산
# - scikit-learn           : cosine_similarity (행렬 연산)
pip install langchain-google-genai numpy scikit-learn
```

`.env`의 `GOOGLE_API_KEY`가 필요합니다(임베딩도 API 호출). 이번 강은 `data/faq.csv` (FAQ 10개)를 씁니다.

1. 단일 문장 임베딩 — `embed_query`

문장 하나를 벡터로 바꿉니다.

```
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent))
from common import get_embeddings

emb = get_embeddings() # GoogleGenerativeAIEmbeddings (gemini-embedding-001, 768차원)
v = emb.embed_query("환불은 며칠 걸려요?") # 문장 한 개 → 벡터

print("벡터 차원:", len(v)) # 768
print("앞 5개 값:", v[:5])
```

예상 출력

```
벡터 차원: 768  
앞 5개 값: [0.0123, -0.0481, 0.0207, 0.0015, -0.0339]
```

`embed_query`는 문장 하나를 768개 숫자로 바꿔 줍니다. 이 숫자들이 그 문장의 의미 좌표입니다.

2. 여러 문장 임베딩 — `embed_documents` (배치)

여러 문장을 한꺼번에 벡터로 바꿀 때는 `embed_documents`를 씁니다.

```
import pandas as pd  
from common import DATA  
  
df = pd.read_csv(DATA / "faq.csv")  
questions = df["question"].tolist() # FAQ 질문 10개  
  
# 여러 문장을 '한 번에' 변환 → 요청 수 · 비용 절감 (백엔드 기본기)  
doc_vectors = emb.embed_documents(questions)  
print("임베딩한 FAQ 수:", len(doc_vectors)) # 10  
print("벡터 차원:", len(doc_vectors[0])) # 768
```

예상 출력

```
임베딩한 FAQ 수: 10  
벡터 차원: 768
```

`embed_documents`는 문장 리스트를 받아 벡터 리스트를 돌려줍니다. 10개 질문 → 10개 벡터죠.

3. embed_query vs embed_documents — 왜 나눠 쓰나

둘 다 텍스트를 벡터로 바꾸는데 왜 나눠 쓸까요?

함수	입력	용도
<code>embed_query</code>	문장 1개	검색할 질문
<code>embed_documents</code>	문장 리스트	인덱싱할 문서들

핵심은 **배치(batch)** 처리입니다. 문서 100개를 임베딩할 때 `embed_query`를 100번 부르면 **API 요청이 100번**입니다. `embed_documents`로 한 번에 보내면 **요청이 1번**(또는 몇 번)으로 줄어 빠르고 쌉니다.

```
# 나쁨 — 100번 요청
vectors = [emb.embed_query(q) for q in questions] # API 100번 호출!

# 좋음 — 1번 요청 (배치)
vectors = emb.embed_documents(questions) # API 1번 호출
```

백엔드 기본기: "여러 개를 처리할 땐 하나씩 말고 한 번에(배치로)." DB에 100개 INSERT를 한 번에 하는 것과 같은 원리입니다. 요청 횟수를 줄이면 빠르고 쌉니다.

4. 벡터를 직접 들여다보기

벡터가 정말 "의미"를 담고 있는지, 두 문장의 벡터를 비교해 봅니다.

```
v1 = emb.embed_query("환불")
v2 = emb.embed_query("돈 돌려받기")
v3 = emb.embed_query("로봇청소기")

# (다음 절에서 코사인 유사도로 비교하지만, 미리 감만)
# v1, v2는 비슷한 숫자 패턴, v3는 다른 패턴일 것
print("환불 앞 3개:", v1[:3])
print("돈돌려받기 앞 3개:", v2[:3]) # v1과 비슷
print("로봇청소기 앞 3개:", v3[:3]) # v1과 다름
```

눈으로 숫자만 봐선 정확히 알기 어렵지만, 코사인 유사도(다음 절)로 재면 "환불"과 "돈 돌려받기"가 가깝고, "로봇청소기"는 멀다는 게 숫자로 나옵니다.

핵심 정리

- 14 강 설치: `pip install langchain-google-genai numpy scikit-learn`.
- `embed_query(문장)` → 벡터 1 개 (검색할 질문용).
- `embed_documents(리스트)` → 벡터 리스트 (인덱싱할 문서용, **배치**).
- 여러 문장은 반드시 `embed_documents` 로 **배치 처리**(요청·비용 절감).

따라하기 — FAQ 임베딩과 유사도 검색

목표

FAQ 질문들을 임베딩해 두고, 사용자 질문과 코사인 유사도로 가장 비슷한 **FAQ**를 찾습니다. 벡터DB 없이 numpy만으로 "미니 의미 검색"을 만듭니다.

1. 준비 — FAQ 임베딩 (05번 복습)

```
import pandas as pd
import numpy as np
from common import get_embeddings, DATA

df = pd.read_csv(DATA / "faq.csv")
questions = df["question"].tolist()

emb = get_embeddings()
doc_vectors = emb.embed_documents(questions) # FAQ 질문들 → 벡터 (배치)
```

FAQ 10개를 미리 벡터로 만들어 뒀습니다. 이게 13강에서 배운 "인덱싱"(사전 준비)입니다.

2. 사용자 질문 임베딩

```
user_q = "돈 언제 돌려받아요?" # '환불'이라는 단어가 전혀 없다!
q_vec = emb.embed_query(user_q) # 질문 한 개는 embed_query로
```

여기서 핵심: **"돈 언제 돌려받아요?"** 에는 **"환불"**이라는 단어가 없습니다. 13강의 단어 매칭이라면 환불 FAQ를 못 찾았겠죠. 임베딩 검색은 어떨까요?

3. 코사인 유사도로 가장 비슷한 FAQ 찾기

03번의 `cosine` 함수로, 질문 벡터와 각 FAQ 벡터의 유사도를 계산합니다.

```
def cosine(a, b) -> float:
    a, b = np.asarray(a, dtype=float), np.asarray(b, dtype=float)
    denom = np.linalg.norm(a) * np.linalg.norm(b)
    return float(a @ b / denom) if denom else 0.0

# 질문 벡터 vs 각 FAQ 벡터의 코사인 유사도를 모두 계산
scores = [cosine(q_vec, dv) for dv in doc_vectors]
best = int(np.argmax(scores)) # 유사도가 가장 높은 FAQ의 인덱스

print(f"질문: {user_q}")
print(f"가장 비슷한 FAQ(유사도 {scores[best]:.3f}):")
print(" Q:", df.iloc[best]["question"])
print(" A:", df.iloc[best]["answer"])
```

예상 출력

```
질문: 돈 언제 돌려받아요?
가장 비슷한 FAQ(유사도 0.812):
Q: 환불은 언제 처리되나요?
A: 반품 상품이 물류센터에 도착한 후 영업일 기준 3일 이내에 환불됩니다 ...
```

단어가 하나도 안 겹치는데 "환불 FAQ"를 정확히 찾았습니다! "돈 돌려받기"와 "환불"의 의미가 가깝다는 걸 임베딩이 알아낸 거죠. 이게 **의미 기반 검색**의 힘입니다. 13강 단어 매칭이 못 한 일을 해냈습니다.

4. 동작 원리 정리

- | | |
|------------------------|---------------------------|
| 1. FAQ 10개 임베딩 (사전) | → 10개 벡터 |
| 2. "돈 돌려받기" 임베딩 | → 질문 벡터 |
| 3. 질문 벡터 vs 10개 벡터 유사도 | → [0.81, 0.34, 0.21, ...] |
| 4. 가장 높은 것 선택 | → "환불은 언제 처리되나요?" (0.81) |

의미 검색 4 단계가 그대로 코드가 됐습니다.

np.argmax(scores)가 "유사도가 가장 높은 인덱스"를 찾아 주죠.

5. 상위 k개 + sklearn으로 빠르게

가장 비슷한 1개만이 아니라 상위 3개를 찾고 싶으면? 그리고 매번 루프 도는 게 느리면?

sklearn의 cosine_similarity로 한 번의 행렬 연산으로 처리합니다.

```
from sklearn.metrics.pairwise import cosine_similarity

# 질문 1개 vs 전체 → 유사도 배열 (행렬 연산, 빠름)
sims = cosine_similarity([q_vec], doc_vectors)[0]

# 유사도 내림차순 정렬 후 상위 3개 인덱스
top_k = sims.argsort()[::-1][:3]

print("Top-3 유사 FAQ:")
for rank, idx in enumerate(top_k, 1):
    print(f" {rank}. ({sims[idx]:.3f}) {df.iloc[idx]['question']}")
```

예상 출력

```
Top-3 유사 FAQ:
1. (0.812) 환불은 언제 처리되나요?
2. (0.659) 주문 취소는 어떻게 하나요?
3. (0.641) 교환은 어떻게 신청하나요?
```

argsort()[::-1][:3]을 뜯어보면:

- argsort(): 유사도를 오름차순으로 정렬한 인덱스
- [::-1]: 뒤집어서 내림차순(높은 것부터)
- [:3]: 상위 3개

"환불"이 1위, 그다음 "취소", "교환" 순으로 의미가 가까운 FAQ들이 나왔습니다. 모두 환불과 관련된 주제죠.

6. numpy 루프 vs sklearn 행렬

두 방법은 같은 결과를 내지만 효율이 다릅니다.

방법	특징
numpy 루프(<code>for</code>)	원리가 잘 보임, 느림
sklearn 행렬(<code>cosine_similarity</code>)	빠름, 깔끔함

문서가 적으면 루프도 괜찮지만, 수천 개면 행렬 연산이 훨씬 빠릅니다. 그리고 수십만 개면? 그땐 행렬 연산도 느려져서 전용 벡터DB(FAISS)가 필요합니다 — 그게 15강입니다.

핵심 정리

- FAQ를 미리 임베딩해 두고, 질문과 코사인 유사도로 검색.
- 단어가 안 겹쳐도("돈 돌려받기") 의미로 찾을 수 있음("환불 FAQ").
- np.argmax로 1위, argsort()[::-1][:k]로 상위 k개.
- cosine_similarity(sklearn)로 빠른 행렬 연산.
- 문서가 아주 많으면 벡터 DB(FAISS)가 필요.

긴 문서 임베딩 전략

긴 문서(직원핸드북 같은)를 통째로 한 벡터에 임베딩하면 안 됩니다.

모델 입력 토큰 한도를 넘고, 책 한 권의 의미가 한 점으로 뭉개져 검색이 부정확해집니다.

청킹 후 청크별로 임베딩해서, 질문과 가장 가까운 조각을 찾습니다.

1. 통째로 임베딩하면 안 되는 이유

"긴 문서도 그냥 임베딩하면 안 되나?"라고 생각할 수 있습니다. 두 가지 문제가 있습니다.

① 토큰 한도 초과

임베딩 모델도 입력에 토큰 한도가 있습니다. 50페이지 핸드북을 통째로 넣으면 한도를 넘어 에러가 나거나 잘립니다.

② 의미가 뭉개짐

더 근본적인 문제입니다. 책 한 권의 의미를 **벡터 한 점**으로 표현하면, "이 책은 대략 인사 규정에 관한 것"이라는 흐릿한 좌표가 됩니다.

직원핸드북 전체 → 벡터 1개

→ "연차 휴가는 며칠?" 질문이 와도

→ 책 전체의 평균 의미와만 비교 → "어느 부분이 답인지" 알 수 없음!

연차 휴가 질문의 답은 핸드북의 특정 부분에 있는데, 통째 벡터로는 그 부분을 짚을 수 없습니다.

비유: 책 한 권을 "이 책은 대충 이런 느낌"이라는 **한 문장 요약**으로만 기억하면, "3장 5절 내용"을 물었을 때 답할 수 없습니다. 페이지별로 기억해야 특정 부분을 짚죠.

2. 해법 — 청킹 후 청크별 임베딩

해법은 13강에서 배운 청킹입니다. 긴 문서를 조각으로 나눈 뒤, 각 조각을 따로 임베딩합니다.

(실습 파일: [sections/14_embedding/07_긴-문서-임베딩-전략.py](#))

`직원핸드북.pdf` 를 이렇게 처리합니다.

1. ① 청킹(13강): `[청크1] [청크2] ... [청크N]` (각 ≤ 500 자)
2. ② 청크별 임베딩: `[벡터1] [벡터2] ... [벡터N]` (각 청크의 의미 좌표)
3. ③ 질문과 가장 가까운 청크 검색 → "연차 휴가 부분" 청크를 정확히 찾음!

이러면 질문이 "연차 휴가 며칠?"일 때, 연차 관련 청크만 콕 집어 찾을 수 있습니다.

3. 코드 — 청킹 + 청크별 임베딩

```
from common import get_embeddings, DOCS
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
import numpy as np

# ① 긴 문서 로드
docs = PyPDFLoader(str(DOCS / "직원핸드북.pdf")).load()
full_text = "\n".join(d.page_content for d in docs)
print(f"전체 길이: {len(full_text):,}자") # 매우 김!

# ② 청킹 (13강)
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)
chunk_texts = [c.page_content for c in chunks]

# ③ 청크별 임베딩 (배치)
emb = get_embeddings()
chunk_vectors = emb.embed_documents(chunk_texts) # 청크들을 한 번에 벡터화
store = list(zip(chunk_texts, chunk_vectors)) # (텍스트, 벡터) 쌍 보관
print(f"청크 {len(store)}개 벡터 보관")
```

청킹과 임베딩이 결합된 모습입니다. 청킹으로 나누고, `embed_documents` 로 청크들을 배치 임베딩하죠.

4. 청크 검색

질문과 가장 가까운 청크를 찾습니다(06번과 같은 원리).

```
from ch14_embedding import cosine # 14강 본편의 코사인 헬퍼 재사용

user_q = "연차 휴가는 며칠까지 쓸 수 있나요?"
q_vec = emb.embed_query(user_q)

# 질문 벡터와 모든 청크 벡터의 유사도 → 가장 높은 청크가 답의 근거
scores = [cosine(q_vec, v) for _, v in store]
order = np.argsort(scores)[::-1][:3] # 상위 3개

print(f"질문: {user_q}")
for rank, idx in enumerate(order, 1):
    text = chunk_texts[idx].replace("\n", " ")
    print(f" {rank}. (유사도 {scores[idx]:.3f}) {text[:90]} ...")
```

예상 출력

```
질문: 연차 휴가는 며칠까지 쓸 수 있나요?
1. (0.784) 제5장 휴가 규정. 연차 유급휴가는 근무 1년당 15일이 부여되며 ...
2. (0.612) 휴직 및 복직 절차는 다음과 같다 ...
3. (0.589) 경조사 휴가는 결혼 5일, 출산 ...
```

핸드북 전체가 아니라, 연차 관련 청크(0.784)를 1위로 정확히 찾았습니다. 통째 벡터였다면 불가능했을 일이죠.

5. 보관 — 메모리 vs 벡터DB

위 코드는 청크 벡터를 `store` (메모리 리스트)에 보관했습니다. 실습에선 괜찮지만, 실무에선 문제가 있습니다.

- **메모리 보관**: 프로그램이 끝나면 사라짐 → 매번 다시 임베딩(비용!)
- **벡터DB 보관**: 디스크에 영속 저장 → 한 번 만들면 재사용

수천~수만 청크를 매번 임베딩하면 시간·비용이 막대합니다. 그래서 **한 번 임베딩한 벡터를 저장해** 두고 재사용해야 하죠. 그게 **벡터 DB(FAISS)** 입니다. 이 강의 메모리 보관은 원리를 보여주는 것이고, 실무는 벡터 DB 로 갑니다.

핵심 정리

- 긴 문서를 **통째로 임베딩하면** 토큰 초과 + 의미 뭉개짐.
- 해법: **청킹 후 청크별 임베딩** → 필요한 조각만 검색.
- 질문과 가장 가까운 청크가 **답의 근거**가 된다.
- 메모리 보관은 임시 — 실무는 **벡터 DB** 로 영속 저장.

3. RAG qa

문서 근거로만 답하고 출처를 밝혀라

2장까지 "관련 청크 k개"를 찾을 수 있게 됐습니다. 하지만 고객에게 청크 덩어리를 던질 순 없죠. 필요한 건 **자연어 답변**입니다. 그것도 **문서에 없으면 지어내지 않고(환각 억제)**, 출처를 함께 밝히는 답변입니다. 이게 RAG QA 시스템입니다.

1. 검색만으로는 부족하다

15강까지의 RAG는 "검색"까지였습니다.

```
results = vs.similarity_search("환불 며칠 걸려?", k=4)
# → [청크1, 청크2, 청크3, 청크4] ← 청크 덩어리!
```

고객에게 이 청크 4개를 그대로 보여 줄 순 없습니다. "환불은 영업일 기준 3일 이내입니다"처럼 **깔끔한 답**이 필요하죠. 검색한 청크를 LLM에게 주고 **답을 만들게** 해야 합니다 — 이게 RAG의 마지막 단계 ⑤ **생성**입니다.

2. 두 가지가 더 필요하다

단순히 "청크를 LLM에게 줘서 답하게" 하면 될까요? 두 가지 문제가 남습니다.

① 환각 억제

LLM은 청크에 없는 내용을 그럴듯하게 **지어낼** 수 있습니다(1강 환각). CS 답변이 틀리면 사고죠.

```
청크에 "환불 3일"만 있는데
LLM이 "환불 3일, 그리고 무료로 반품 픽업도 해드립니다" 라고 덧붙이면? → 거짓 정보!
```

② 출처 표시

답변이 어느 문서 몇 페이지 근거인지 보여야 신뢰가 갑니다. 감사·디버깅에도 필수죠.

"환불은 3일 이내입니다. (출처: 환불교환정책.pdf 1페이지)"

괄호 안 출처가 있어야 답을 믿을 수 있습니다.

3. RAG QA = 검색 + 제약 + 출처

이 셋을 한 번에 푸는 게 RAG QA 체인입니다.

질문 이 들어오면:

1. ① 검색(15강) — 관련 청크 k개
2. ② 프롬프트 조립 — "이 문서 근거로만 답하라"
3. ③ 생성 — LLM이 답변
4. ④ 출처 첨부 — 어느 문서 몇 페이지

→ 결과: `{"answer": "환불은 3일 이내", "sources": [{"환불교환정책.pdf", 1}]}`

핵심은 "검색만 붙인다고 RAG가 아니라, 검색 결과 안에서만 답하라는 제약"이 RAG를 RAG답게 만든다는 것입니다.

핵심 정리

- 검색한 청크를 LLM에게 줘 **자연어 답변**으로 만든다(⑤ 생성).
- 두 가지 필수: **환각 억제**("문서 없으면 모른다") + **출처 표시**.
- RAG QA = **검색** + "**근거로만 답하라**" **제약** + **출처**.
- `answer(q)` -> dict로 모듈화해 백엔드에 바로 연결.

QA 체인 구조 (LCEL)

LCEL(LangChain Expression Language) 은 |파이프|로 단계를 잇는 방식입니다. RAG QA 체인은 검색 → 프롬프트 조립 → LLM → 문자열화를 한 줄로 엮습니다. 유닉스 파이프(|)처럼 데이터가 단계를 따라 흐릅니다.

1. QA 체인의 흐름

RAG QA는 네 단계가 차례로 흐릅니다.

질문 이 네 단계를 차례로 흐릅니다.

1. 검색 → `retriever` (청크 k개 → format → context)
2. 조립 → 프롬프트 ("문서 근거로만 답하라" + context + 질문)
3. 생성 → LLM (답변 생성)
4. 변환 → `StrOutputParser` (문자열로) → 최종 답변

각 단계의 출력이 다음 단계의 입력이 됩니다. 이걸 코드로 깔끔하게 잇는 게 LCEL입니다.

2. LCEL — 파이프로 잇기

LCEL은 `|` 기호로 단계를 연결합니다.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | PROMPT          # 프롬프트 조립
    | llm             # LLM 생성
    | StrOutputParser() # 문자열로 변환
)

answer = rag_chain.invoke("환불 며칠 걸려?")
```

`|`로 이어진 이 한 줄이 곧 "검색 → 조립 → 생성 → 문자열화" 전체입니다.

비유: 유닉스 셸의 파이프와 똑같습니다. `cat file | grep "환불" | sort` 처럼 데이터가 `|`를 따라 흐르죠. LCEL도 질문이 `|`를 따라 흘러가며 검색·생성을 거칩니다. 한 줄로 파이프라인을 표현하는 거죠.

3. 첫 단계 뜯어보기

가장 헛갈리는 첫 단계를 자세히 봅시다.

```
{"context": retriever | format_docs, "question": RunnablePassthrough()}
```

이건 딕셔너리인데, 두 키를 채웁니다.

- **"context"**: `retriever | format_docs`
질문을 `retriever`에 넣어 청크를 검색하고, `format_docs`로 하나의 문자열로 합칩니다. → 프롬프트의 `{context}` 자리에 들어갈 값
- **"question"**: `RunnablePassthrough()`
입력(질문)을 그대로 흘려보냅니다. → 프롬프트의 `{question}` 자리에 들어갈 값

즉, 질문 하나가 들어오면 → context(검색 결과)와 question(원본 질문)을 동시에 만들어, 프롬프트의 두 빈칸을 채웁니다.

4. RunnablePassthrough란?

`RunnablePassthrough()`는 이름 그대로 입력을 그대로 통과시키는 부품입니다.

```
"question": RunnablePassthrough()  
# 질문이 들어오면 → 가공 없이 그대로 {question} 자리로
```

`context`는 검색·포맷 가공이 필요하지만, `question`은 원본 질문 그대로 프롬프트에 넣어야 합니다. 그래서 "아무것도 하지 말고 통과시켜"라는 `RunnablePassthrough()`를 씁니다. 데이터를 손대지 않고 다음으로 넘기는 거죠.

5. format_docs — 청크를 문자열로

`retriever`가 돌려주는 건 `Document` 리스트입니다. 하지만 프롬프트에는 문자열이 들어가야 하죠. 그래서 `format_docs`로 변환합니다.

```
def format_docs(docs):
    """검색된 Document들의 본문을 한 덩어리 문자열(context)로 합친다."""
    return "\n\n".join(d.page_content for d in docs)
```

청크 4개의 `page_content`를 줄바꿈 두 개로 이어 붙여 하나의 긴 문자열을 만듭니다. 이게 프롬프트의 `{context}` 자리에 들어가는 "근거 문서"입니다.

6. LCEL의 장점

"그냥 함수로 차례로 호출하면 안 되나?"라고 생각할 수 있습니다. LCEL을 쓰면 좋은 점이 있습니다.

- 선언적: 파이프라인 구조가 한눈에 보임 (검색→프롬프트→LLM→파서)
- 재사용: 만든 체인(`rag_chain`)을 여러 질문에 그대로 `invoke`
- 스트리밍·배치 지원: 같은 체인으로 `stream`, `batch`도 가능
- 조립 가능: 부품을 갈아끼우기 쉬움 (프롬프트만 교체 등)

LangChain의 권장 방식이고, 일단 익숙해지면 매우 편합니다. 처음엔 `!`가 낯설지만, "데이터가 흐른다"는 직관만 잡으면 됩니다.

핵심 정리

- LCEL = |로 단계를 잇는 방식(유닉스 파이프처럼).
- RAG QA 체인: {context, question} | 프롬프트 | LLM | 파서.
- retriever | format_docs → 검색 후 문자열로(=context).
- RunnablePassthrough() → 질문을 그대로 통과(=question).
- format_docs → 청크 리스트를 하나의 문자열로.

환각을 억제하는 프롬프트

RAG의 핵심은 검색이 아니라 "검색 결과 안에서만 답하라"는 제약입니다.

프롬프트에 "문서 근거로만, 없으면 모른다고, 추측하지 말라"를 명시하면 LLM이 함부로 지어내지 못합니다. 이게 RAG와 일반 챗봇을 가르는 결정적 차이입니다.

1. 검색만으로는 환각을 못 막는다

청크를 검색해 LLM에게 줘도, 그냥 주면 환각이 생길 수 있습니다.

```
# 나쁜 프롬프트 — 그냥 답하라고 하면
"다음 문서를 참고해서 질문에 답해줘: {context}\n질문: {question}"
# → LLM이 문서 + 자기 지식을 섞어 답함 → 문서에 없는 내용도 덧붙일 수 있음 (환각!)
```

"참고해서"라고만 하면, LLM이 문서를 참고하되 자기 지식(환각 가능)도 섞어 답합니다. 그러면 RAG를 쓰는 의미가 없죠. 문서에 없는 거짓이 섞이니까요.

2. 환각 억제 프롬프트

해결책은 프롬프트에 **명확한 제약**을 거는 것입니다.

```
PROMPT = ChatPromptTemplate.from_template(
    "너는 승승장구물 CS 상담원이다.\n"
    "아래 [문서] 내용만 근거로 한국어로 정확히 답하라.\n"
    "문서에 없는 내용은 추측하지 말고 '제공된 문서에서 찾을 수 없습니다'라고 답하라.\n\n"
    "[문서]\n{context}\n\n[질문] {question}\n\n[답변]"
)
```

세 가지 제약이 들어 있습니다.

규칙	효과
"아래 문서 내용만 근거로 답하라"	외부 지식(환각) 사용 차단
"문서에 없으면 '찾을 수 없습니다'라고 답하라"	지어내기 방지
"추측하지 말라"	과신 답변 억제

이 세 줄이 환각을 크게 줄입니다. "문서 안에서만" 놀라고 가뒀 두는 거죠.

3. 왜 이게 RAG의 핵심인가

많은 사람이 "RAG = 검색 붙이기"라고 오해합니다. 하지만 진짜 핵심은 이 **제약**입니다.

검색만 붙임: 문서 검색 + LLM 자유 답변 → 환각 여전히 가능
 검색 + 제약: 문서 검색 + "문서 안에서만 답" → 환각 억제 ✓

검색은 "근거를 찾아 주는 것"이고, 제약은 "그 근거 안에서만 답하게 하는 것"입니다. 둘 다 있어야 RAG가 제대로 작동합니다. 검색만 있고 제약이 없으면, 비싼 검색을 해 놓고 LLM이 무시하고 지어낼 수 있죠.

비유: 오픈북 시험에서 "교재만 보고 답하세요"라는 규칙이 있어야 합니다. 교재를 펴 놓고 (검색) "아무거나 써도 돼요"(제약 없음)라고 하면, 학생이 교재를 무시하고 자기 기억(환각)으로 쓸 수 있죠. "교재 내용만"이라는 규칙(제약)이 핵심입니다.

4. {context}와 {question} 자리

프롬프트의 `{context}`와 `{question}`은 빈칸(**placeholder**)입니다. `invoke` 할 때 실제 값으로 채워집니다.

```
PROMPT.invoke({
  "context": "환불은 영업일 기준 3일 이내 ...", # {context} 자리에
  "question": "환불 며칠 걸려?"                # {question} 자리에
})
```

`{context}`에는 검색한 문서가, `{question}`에는 사용자 질문이 들어갑니다. 02번의 LCEL 첫 단계(`{"context": ..., "question": ..."}`)가 이 두 빈칸을 채우는 거였죠.

5. 제약의 한계와 보완

이 프롬프트 제약은 강력하지만 **100%는 아닙니다**. LLM이 가끔 규칙을 어길 수 있죠. 그래서 실무에서는 추가 장치를 둡니다.

- 출처 표시(04번): 답의 근거를 보여 줘 사람이 검증 가능하게
- 검색 품질(08번): 애초에 좋은 청크를 찾아 줘야 좋은 답
- 온도 0: `temperature=0`으로 일관·보수적 답변

특히 출처 표시가 중요합니다. 답이 틀려도 "이 문서 근거"라고 보여 주면, 사람이 "어, 이 문서엔 그런 말 없는데?"라고 잡아낼 수 있으니까요.

핵심 정리

- RAG의 핵심은 검색이 아니라 "**검색 결과 안에서만 답하라**"는 제약.
- 프롬프트 3제약: "**문서만 근거**" + "**없으면 모른다**" + "**추측 금지**".
- 검색만 붙이면 환각 여전 → **제약**이 있어야 RAG가 작동.
- `{context}`(문서)와 `{question}`(질문)은 `invoke` 시 채워지는 빈칸.
- 제약은 강력하지만 100%는 아니라 **출처 표시**로 보완.

출처를 함께 반환하기

답변만 주면 출처를 잃습니다. 그래서 retriever로 먼저 문서를 가져와 답변과 함께 묶어 반환합니다. 형태는 백엔드 친화적인 dict로 통일합니다 — {"answer": ..., "sources": [...]}.

1. 왜 출처가 필요한가

RAG의 큰 장점 중 하나가 출처 추적 가능합니다(13강). 답변에 출처를 붙이면:

- 신뢰성: "환불교환정책.pdf 1페이지 근거"라고 하면 믿을 만함
- 검증: 사람이 그 문서를 펴서 확인 가능 (환각 잡기)
- 디버깅: 답이 이상하면 "어느 문서를 봤나"를 추적
- 감사(audit): "이 답변의 근거가 무엇이었나" 기록

일반 챗봇은 출처를 못 줍니다("그냥 제 지식입니다"). RAG는 "이 문서에서 나왔습니다" 라고 말할 수 있죠. 이게 기업에서 RAG를 쓰는 큰 이유입니다.

2. 문제 — LCEL 체인은 답변만 준다

02번의 LCEL 체인은 답변 문자열만 돌려줍니다.

```
rag_chain.invoke("환불 며칠 걸려?")  
# → "환불은 영업일 기준 3일 이내입니다." ← 답변만! 출처는 어디로?
```

체인이 **검색 → 프롬프트 → LLM → 문자열**로 흐르면서, 중간의 검색 결과(어느 문서) 정보가 마지막엔 사라집니다. 답변만 남죠. 출처를 잃은 겁니다.

3. 해법 — 검색을 따로 먼저

출처까지 주려면, retriever로 문서를 먼저 뽑아 두고 답변과 함께 묶습니다.

```
def answer(question: str) -> dict:
    """질문 -> {answer, sources}."""
    docs = retriever.invoke(question) # ① 검색 (문서를 따로 보관!)
    context = format_docs(docs)       # ② 조립
    # ③ 생성
    text = (PROMPT | llm | StrOutputParser()).invoke(
        {"context": context, "question": question}
    )
    # ④ docs에서 출처 추출 (위에서 보관해 둔 docs를 재사용)
    sources = extract_sources(docs)
    return {"answer": text, "sources": sources}
```

핵심은 `docs = retriever.invoke(question)` 을 먼저 해서 `docs` 를 변수에 잡아 두는 것입니다. 그러면 답변 생성 후에도 `docs` 로 출처를 뽑을 수 있죠. 검색을 체인 안에 숨기지 않고 밖으로 꺼낸 겁니다.

4. 출처 추출 + 중복 제거

검색된 청크에서 출처(파일명·페이지)를 뽑되, 중복을 제거합니다. 같은 페이지에서 여러 청크가 올 수 있거든요.

```
uniq, seen = [], set()
for d in docs:
    src = d.metadata.get("source", "?").split("/")[-1] # 파일명만
    page = d.metadata.get("page")
    key = (src, page)
    if key not in seen: # 중복 제거
        seen.add(key)
        uniq.append({"source": src, "page": page})
```

- `d.metadata.get("source")`: 13강에서 보존된 출처(전체 경로) → `.split("/")[-1]` 로 파일명만
- `d.metadata.get("page")`: 페이지 번호
- `seen` set으로 같은 (파일, 페이지) 조합을 한 번만

청크 4개가 모두 1페이지에서 왔다면, 출처는 [{환불교환정책.pdf, 1}] 하나로 정리됩니다.

5. dict 반환 — 백엔드 친화적

`answer`가 `dict`를 돌려주는 게 포인트입니다.

```
{"answer": "환불은 3일 이내입니다.", "sources": [{"source": "환불교환정책.pdf", "page": 1}]}
```

이 dict는 **JSON**으로 바로 변환됩니다. 그래서 FastAPI/Flask 같은 웹 서버에서 그대로 반환할 수 있죠.

```
# FastAPI 핸들러에서
@app.post("/qa")
def qa(question: str):
    return answer(question)  # dict가 자동으로 JSON 응답이 됨!
```

답변과 출처를 dict로 묶어 두니, 프론트엔드에서 "답변은 본문에, 출처는 하단에 작게" 표시하는 식으로 쓰기 좋습니다. 06번에서 이 모듈화를 완성합니다.

핵심 정리

- 출처는 신뢰성·검증·디버깅·감사에 필수(RAG의 큰 장점).
- LCEL 체인은 답변만 줘서 출처를 잃음.
- 해법: retriever로 문서를 먼저 뽑아 답변과 함께 묶기.
- 출처는 파일명·페이지 추출 + 중복 제거(set).
- {"answer": ..., "sources": [...]} dict로 반환 → 백엔드 친화적.

따라하기 — QA 체인 만들기 (LCEL)

목표

retriever + 프롬프트 + LLM을 LCEL로 엮어 RAG QA 체인을 만듭니다. 실제 파일은 `code/ch16_rag_qa.py` 입니다.

0. 실습 준비 (설치)

```
conda activate agentic

# 16강에 필요한 라이브러리 (앞 강에서 설치했다면 생략 가능)
pip install langchain langchain-community langchain-google-genai faiss-cpu pypdf
```

이번 실습은 data/docs/환불교환정책.pdf, 멤버십정책.pdf를 씁니다. API_KEY가 필요합니다.

1. 인덱싱 — 두 문서 통합 retriever

15강의 인덱싱으로 retriever를 만듭니다. 두 정책 문서를 한 인덱스에 통합합니다.

```
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent))
from common import get_chat, get_embeddings, DOCS
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
```

```
def build_retriever():
    """정책 문서 2개를 통합 인덱싱한 retriever 반환."""
    docs = []
    for f in ["환불교환정책.pdf", "멤버십정책.pdf"]:
        docs.extend(PyPDFLoader(str(DOCS / f)).load()) # 두 문서를 한 인덱스로
    chunks = RecursiveCharacterTextSplitter(
        chunk_size=500, chunk_overlap=50).split_documents(docs)
    vs = FAISS.from_documents(chunks, get_embeddings())
    # as_retriever: 벡터스토어를 'k개를 찾아 주는 검색 객체'로 변환
    return vs.as_retriever(search_kwargs={"k": 4})
```

`as_retriever`가 새 얼굴입니다. 벡터스토어를 "질문을 주면 청크 k개를 찾아 주는 검색 객체 (retriever)"로 바꿔 줍니다. `search_kwargs={"k": 4}`는 "상위 4개를 찾아라"는 뜻이죠.

2. retriever란?

`retriever`는 LCEL 체인에 끼우기 좋은 검색 전용 객체입니다.

```
retriever = build_retriever()
docs = retriever.invoke("환불 며칠 걸려?") # 질문 → 관련 청크 4개
```

15강의 `vs.similarity_search(query, k=4)`와 같은 일을 하지만, `retriever.invoke(질문)` 형태라 LCEL 체인(**!**)에 바로 연결할 수 있습니다. 그래서 RAG 체인을 만들 땐 `as_retriever`로 retriever를 씁니다.

3. 프롬프트와 format_docs

03번의 환각 억제 프롬프트와 02번의 format_docs를 준비합니다.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
```

```
PROMPT = ChatPromptTemplate.from_template(
    "너는 승승장구몰 CS 상담원이다.\n"
    "아래 [문서] 내용만 근거로 한국어로 정확히 답하라.\n"
    "문서에 없는 내용은 추측하지 말고 '제공된 문서에서 찾을 수 없습니다'라고 답하라.\n\n"
    "[문서]\n{context}\n\n[질문] {question}\n\n[답변]"
)

def format_docs(docs):
    """검색된 Document들의 본문을 한 덩어리 문자열(context)로 합친다."""
    return "\n\n".join(d.page_content for d in docs)
```

4. LCEL 체인 조립

이제 모든 부품을 `!`로 잇습니다.

```
llm = get_chat(temperature=0) # temperature=0: 같은 질문에 일관된 답(재현성)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | PROMPT
    | llm
    | StrOutputParser()
)

print(rag_chain.invoke("환불 며칠 걸려?"))
```

예상 출력

반품 상품이 물류센터에 도착한 후 영업일 기준 3일 이내에 환불됩니다.
카드 취소는 카드사 사정에 따라 추가 시간이 소요될 수 있습니다.

02번에서 본 그 LCEL 체인입니다. 질문 하나가:

1. `retriever | format_docs` → 검색 후 context 문자열로
2. `RunnablePassthrough()` → 질문 그대로
3. `PROMPT` → 두 빈칸 채워 프롬프트 완성
4. `llm` → 답변 생성
5. `StrOutputParser()` → 문자열로

5. temperature=0인 이유

`get_chat(temperature=0)` 을 썼습니다. 왜 일까요?

RAG QA는 사실 기반 답변입니다. 같은 질문엔 항상 같은 답이 나와야 신뢰가 가죠.

`temperature=0` 은 무작위성을 꺼서 일관된(재현 가능한) 답변을 보장합니다(2강 복습). 창작이 아니라 정확한 정보 전달이 목적이니까요.

홍보 문구 → temperature 0.7 (다양성)

RAG QA → temperature 0 (일관성 · 정확성)

핵심 정리

- `as_retriever(search_kwargs={"k": 4})` — 벡터스토어를 검색 객체로 변환.
- `retriever.invoke(질문)`은 LCEL 체인()에 바로 연결 가능.
- LCEL: `{context, question} | PROMPT | llm | StrOutputParser()`.
- RAG QA는 **temperature=0**(일관·정확).

따라하기 — 답변과 출처를 함께: `answer()` 함수로 모듈화

목표

04번에서 설계한 대로, 답변과 출처를 함께 반환하는 `answer(question) -> dict` 함수를 만듭니다. 백엔드 핸들러에 바로 쓸 수 있는 형태로요.

1. 서비스 객체는 1회만 생성 (15강 원칙)

`retriever`와 `llm`을 만드는 건 비쌉니다(인덱싱·인증). 요청마다 만들면 안 되죠. 전역에 캐시해 재 사용합니다.

```
# 서비스 객체는 1회만 생성해 재사용
_retriever = None
_llm = None

def _ensure():
    """retriever·llm을 최초 1회만 생성(지연 초기화). 이후는 재사용."""
    global _retriever, _llm
    if _retriever is None:
        _retriever = build_retriever()
        _llm = get_chat(temperature=0)
```

`_ensure()` 는 지연 초기화(**lazy init**) 패턴입니다. 처음 호출될 때만 retriever·llm을 만들고, 이후엔 만들어 둔 걸 재사용합니다. 15강의 "인덱싱은 1회, 서비스는 검색만" 원칙을 코드로 구현한 거죠.

2. answer 함수

04번에서 설계한 `answer` 함수입니다. 검색을 먼저 하고, 답변 생성 후, 출처를 뭉칩니다.

```
def answer(question: str) -> dict:
    """질문 -> {answer, sources}. FastAPI/Flask 핸들러에서 그대로 return 가능."""
    _ensure()
    docs = _retriever.invoke(question) # ① 검색: 근거 청크 확보 (docs 보관!)
    context = format_docs(docs) # ② 조립: 청크를 context 문자열로
    # ③ 생성: 프롬프트 | LLM | 파서를 LCEL로 한 번에 실행
    text = (PROMPT | _llm | StrOutputParser()).invoke(
        {"context": context, "question": question}
    )
    # ④ 출처 추출 + 중복 제거 (04번)
    uniq, seen = [], set()
    for d in docs:
        src = d.metadata.get("source", "?").split("/")[-1]
        page = d.metadata.get("page")
        key = (src, page)
        if key not in seen:
            seen.add(key)
            uniq.append({"source": src, "page": page})
    return {"answer": text, "sources": uniq}
```


핵심은 docs를 먼저 잡아 두는 것(①)입니다. 그래야 답변 생성 후에도 docs로 출처를 뽐쥌(④).

3. 실행

```
res = answer("VIP 등급 조건은?")
print("답변:", res["answer"])
print("출처:")
for s in res["sources"]:
    print(f"  - {s['source']} p.{s['page']}")
```

예상 출력

```
답변: VIP 등급은 최근 6개월 누적 구매금액 100만원 이상일 때 부여됩니다 ...
출처:
  - 멤버십정책.pdf p.0
  - 환불교환정책.pdf p.1
```

답변과 함께 어느 문서 몇 페이지 근거인지 나왔습니다. 고객은 답변을 보고, 의심되면 출처 문서를 확인할 수 있죠. 신뢰가 가는 답변입니다.

4. 백엔드 핸들러에 바로 연결

`answer`가 `dict`를 반환하므로, 웹 서버에 바로 꽂을 수 있습니다.

```
# FastAPI 예시 (29강에서 제대로 다룸)
from fastapi import FastAPI
app = FastAPI()

@app.post("/qa")
def qa_endpoint(question: str):
    return answer(question)  # dict → JSON 응답 자동 변환!
```

`answer(question)`이 돌려주는 dict가 그대로 **JSON** 응답이 됩니다. 프론트엔드는 `answer` 필드를 본문에, `sources` 필드를 출처 표시에 쓰면 되죠. 이렇게 모듈화하면 RAG QA를 실제 API로 만들기 쉽습니다.

5. 모듈화의 가치

`answer` 함수 하나로 RAG의 모든 단계를 감쌌습니다.

`answer(question)` 한 함수가 네 단계를 처리합니다.

1. ① 검색 (retriever)
2. ② 조립 (format_docs)
3. ③ 생성 (LCEL 체인)
4. ④ 출처 (metadata)

→ 반환: `{answer, sources}`

복잡한 RAG 파이프라인을 함수 하나로 추상화한 거죠. 이걸 쓰는 쪽(API 핸들러)은 내부 복잡성을 몰라도 `answer(q)` 만 부르면 됩니다. 좋은 모듈화의 예입니다 — 복잡한 걸 단순한 인터페이스 뒤에 숨기는 것.

핵심 정리

- retriever-llm은 1회 생성 후 재사용(_ensure 지연 초기화).
- answer(q): ① 검색(docs 보관) → ② 조립 → ③ 생성 → ④ 출처 묶기.
- 반환은 {"answer": ..., "sources": [...]} dict.
- dict라서 FastAPI/Flask에 바로 연결(JSON 자동 변환).
- 복잡한 RAG를 함수 하나로 모듈화 → 쓰기 쉬움.

따라하기 — 모르는 질문엔 모른다고

목표

환각 억제 프롬프트(03번)가 실제로 작동하는지 확인합니다. 문서에 없는 내용을 물었을 때, 에이전트가 지어내지 않고 "모른다"고 답하는지 봅니다.

1. 문서에 없는 질문 던지기

환불·멤버십 문서에는 없는 내용을 물어봅니다.

```
print(answer("당일 새벽배송 되나요?")["answer"])
```

"당일 새벽배송"은 환불 정책·멤버십 정책 문서에 없는 내용입니다. (배송 정책 문서가 따로 있다면 거기 있겠지만, 우리 인덱스엔 없죠.) 일반 챗봇이라면 "네, 새벽배송 가능합니다" 같은 그럴듯한 거짓을 지어낼 수 있습니다.

예상 출력

제공된 문서에서 찾을 수 없습니다.

지어내지 않고 "찾을 수 없습니다" 라고 정직하게 답했습니다! 이게 03번 환각 억제 프롬프트가 의도한 동작입니다.

2. 왜 이게 중요한가

"모른다고 답하는 게 뭐가 대단해?"라고 생각할 수 있습니다. 하지만 이건 **매우** 중요합니다.

나쁜 답: "당일 새벽배송 가능합니다!" (지어냄) → 고객이 믿고 주문 → 배송 안 됨 → 사고!
좋은 답: "제공된 문서에서 찾을 수 없습니다" → 고객이 다른 경로로 확인 → 안전

CS에서 틀린 정보를 자신 있게 주는 것은 모른다고 하는 것보다 훨씬 위험합니다. 고객이 그 거짓을 믿고 행동하니까요. "모른다"는 답은 정직하고, 사고를 막습니다.

비유: 의사가 모르는 병에 "괜찮을 거예요"라고 둘러대면 위험합니다. "정확히 모르겠으니 검사를 해 봅시다"가 맞죠. AI도 모르면 모른다고 해야 안전합니다.

3. 정상 질문과 비교

같은 시스템에 문서에 있는 질문을 던지면 잘 답합니다.

```
print(answer("환불 며칠 걸려?")["answer"])
# → "반품 상품이 물류센터 도착 후 영업일 기준 3일 이내에 환불됩니다." (문서 근거!)

print(answer("당일 새벽배송 되나요?")["answer"])
# → "제공된 문서에서 찾을 수 없습니다." (문서에 없음)
```

같은 에이전트가:

- 문서에 있으면 → 정확히 답하고
- 문서에 없으면 → 모른다고 합니다

이 "있으면 답, 없으면 모른다"의 구분이 RAG QA의 핵심 동작입니다. 문서 범위 안에서만 신뢰할 수 있는 답을 주는 거죠.

4. "모른다" 다음의 폴백

실무에서는 "모른다"로 끝내지 않고, 다음 행동을 안내하면 더 좋습니다(12강 폴백 정신).

```
PROMPT_WITH_FALLBACK = ChatPromptTemplate.from_template(
    "너는 승승장구몰 CS 상담원이다.\n"
    "아래 [문서] 내용만 근거로 답하라.\n"
    "문서에 없으면 '해당 내용은 확인이 어렵습니다. 고객센터(1588-0000)로 문의해 주세요'라고 답하라"
    "[문서]\n{context}\n\n[질문] {question}\n\n[답변]"
)
```

이러면 "당일 새벽배송 되나요?"에 "확인이 어렵습니다. 고객센터로 문의해 주세요"라고 안내합니다. 단순히 "모른다"보다 다음 경로를 제시하니 고객 경험이 낫죠. 9·12강에서 배운 폴백을 RAG에 적용한 것입니다.

5. 검증 — 자동 채점

RAG QA가 제대로 작동하는지 자동으로 검증할 수 있습니다. 정답이 정해진 질문으로 채점하는 거죠(3강 정확도 측정의 RAG 버전).

```
test_cases = [
    ("환불 며칠 걸려?", "3일"),          # 문서에 있음 → "3일" 포함해야
    ("당일 새벽배송 되나요?", "찾을 수 없"), # 문서에 없음 → "찾을 수 없" 포함해야
]

passed = sum(1 for q, expected in test_cases if expected in answer(q)["answer"])
print(f"통과: {passed}/{len(test_cases)}")
```

이렇게 "있는 질문엔 정답을, 없는 질문엔 모른다를" 자동 검증하면, 프롬프트나 검색을 바꿨을 때 품질이 유지되는지 확인할 수 있습니다.

핵심 정리

- 문서에 **없는** 질문엔 지어내지 않고 **"찾을 수 없습니다"** 라고 답함(03번 프롬프트 효과).
- **틀린 정보를 자신 있게** 주는 게 모른다보다 위험 — 정직이 안전.
- 같은 에이전트가 "있으면 답, 없으면 모른다"를 구분.
- 실무는 "모른다" + **다음 경로 안내**(폴백)가 더 좋다.
- **자동 채점**으로 RAG 품질을 검증할 수 있다.

검색 품질 높이기 — k값과 재정렬(rerank)

RAG 답변의 품질은 검색 품질에 달려 있습니다. k(검색 개수)를 늘리면 누락은 줄지만 노이즈가 섞입니다. 그래서 많이 뽑은 뒤(k=6) LLM이 관련도로 점수를 매겨 상위만 추리는 재정렬(rerank)을 적용합니다.

1. "검색이 나쁘면 답도 나쁘다"

RAG는 검색한 청크를 근거로 답합니다. 그러니 검색이 엉뚱한 청크를 가져오면, 답도 엉뚱합니다.

좋은 검색: 질문에 딱 맞는 청크 → LLM이 정확히 답
나쁜 검색: 무관한 청크 → LLM이 근거 없이 해매거나 "모른다"

그래서 RAG 품질을 높이려면 검색 품질부터 높여야 합니다. "쓰레기를 넣으면 쓰레기가 나온다"는 11강 격언이 여기서도 통하죠.

2. k값의 딜레마

k는 "몇 개의 청크를 가져올지"입니다. 이게 딜레마를 만듭니다.

k가 작으면 (k=2):

- 관련 청크를 놓칠 수 있음 (누락)
- 답에 필요한 정보가 빠질 위험

k가 크면 (k=6):

- 누락은 줄지만 무관한 청크도 섞임 (노이즈)
- LLM이 헛갈리거나 엉뚱한 부분 참고

작으면 놓치고, 크면 노이즈가 깎입니다. 어느 쪽도 완벽하지 않죠. (실습 파일:

[sections/16_rag_qa/08_검색-품질-높이기-k값과-재정렬.py](#))

```
small = get_retriever(2).invoke(question) # k=2: 적게  
large = get_retriever(6).invoke(question) # k=6: 많이  
print(f"k=2: {len(small)}건 / k=6: {len(large)}건")
```

3. 해법 — 재정렬(rerank)

영리한 해법이 있습니다. 많이 뽑은 뒤($k=6$), 다시 추리는 것입니다.

1. $k=6$ 으로 넉넉히 검색 (누락 방지)
2. LLM이 각 청크의 질문 관련도를 0~10점으로 채점
3. 상위 2개만 추려 LLM에게 근거로 줌 (노이즈 제거)

"넓게 뽑고 → 정밀하게 추리는" 2단계입니다. 그물을 크게 던져 많이 잡은 뒤, 좋은 것만 골라내는 거죠.

4. rerank 구현

LLM에게 각 청크의 관련도를 점수로 매기게 합니다.

```
def rerank(question, docs, top_n=2):
    """검색된 청크들을 LLM이 질문 관련도(0~10)로 채점해 상위 top_n만 재선택한다."""
    llm = get_chat(temperature=0.0)
    scored = []
    for d in docs:
        prompt = (
            "다음 [문서조각]이 [질문]에 답하는 데 얼마나 관련 있는지 0~10 점수로만 답하라.\n"
            "오직 숫자 하나만 출력하라(설명 금지).\n\n"
            f"[질문] {question}\n\n[문서조각]\n{d.page_content[:400]}\n\n[점수]"
        )
        score = _parse_score(llm.invoke(prompt).content)
        scored.append((score, d))
    scored.sort(key=lambda x: x[0], reverse=True) # 점수 내림차순
    return scored[:top_n]                        # 상위 top_n
```

각 청크마다 "이 청크가 질문에 얼마나 관련 있나?"를 LLM에게 0~10점으로 물어, 높은 점수 순으로 상위 2개만 남깁니다.

5. 왜 LLM 재정렬이 더 정밀한가

"벡터 유사도로 이미 정렬했는데 왜 또?"라고 생각할 수 있습니다.

벡터 유사도: 빠르지만 '거칠다' (의미 좌표의 거리만 봄)

LLM 재정렬: 느리지만 '정밀하다' (질문과 청크를 실제로 읽고 판단)

벡터 유사도는 빠른 대신 거칩니다 — "대충 비슷한 방향"을 봅니다. LLM 재정렬은 청크를 실제로 읽고 "이게 질문에 답이 되나?"를 판단하니 더 정밀하죠. 대신 LLM 호출이라 느리고 비쌌습니다.

비유: 도서관에서 책을 찾을 때, 서가 분류(벡터 검색)로 후보를 빠르게 추리고, 직접 펴 보며(LLM 재정렬) 진짜 필요한 책을 고르는 것과 같습니다. 분류만으로는 거칠고, 다 펴 보면 느리니, 둘을 결합하죠.

6. 재정렬의 트레이드오프

재정렬은 공짜가 아닙니다.

- 장점: 검색 정확도 향상 → RAG 답변 품질 향상
- 단점: 청크마다 LLM 호출 → 비용·지연 증가

그래서 항상 쓰는 게 아니라, 정확도가 중요한 경우에 씁니다. 빠른 응답이 중요하면 벡터 검색만, 정확도가 중요하면 재정렬을 추가하는 식이죠. (실무에는 전용 rerank 모델도 있어, LLM보다 싸고 빠르게 재정렬하기도 합니다.)

7. 검색 품질을 높이는 다른 방법들

재정렬 외에도 검색 품질을 높이는 방법이 있습니다.

- 청크 크기 튜닝(13강): 적절한 `chunk_size`로 검색 단위 최적화
- 메타데이터 필터(15강): 관련 문서로 범위를 좁혀 노이즈 제거
- 쿼리 재작성: 질문을 검색에 좋은 형태로 바꾸기
- 하이브리드 검색: 의미 검색 + 키워드 검색 결합

RAG 품질 개선은 "검색을 어떻게 더 정확하게"의 끝없는 여정입니다. 측정하고(자동 채점) 개선하는 순환이죠

핵심 정리

- RAG 답변 품질은 **검색 품질**에 달렸다("검색 나쁘면 답도 나쁨").
- k 딜레마: 작으면 **누락**, 크면 **노이즈**.
- **재정렬(rerank)**: 많이 뽑고($k=6$) → LLM이 관련도 점수 → 상위만 추림.
- LLM 재정렬은 벡터 검색보다 **정밀**(실제로 읽고 판단)하지만 비쌘.
- 검색 품질 개선은 청크 튜닝·필터·쿼리 재작성 등 다양.